Software de Aerolíneas Low-Cost

Juan Molinas

Escuela Técnica N°32 DE 14 “Gral. José de San Martin”, CABA

Proyecto Interdisciplinario

Nahuel Acosta y

Gabriel Abello Tuinyla

CABA, Argentina

6 de Agosto de 2026

La presente carpeta de campo tiene como objetivo registrar de manera sistemática y detallada las actividades realizadas por Juan Molinas en el marco de las unidades curriculares: Programación Orientada a Objetos, Redes, Análisis de Sistemas, Base de Datos y Proyecto Informático II. El propósito principal es consolidar un control organizado de las tareas individuales y grupales, documentando con rigurosidad el proceso de desarrollo, los hitos alcanzados y las resoluciones técnicas ante las dificultades encontradas.

Cada entrada de la bitácora presentará una estructura normalizada que incluirá: fecha, localización, franja horaria de ejecución y una descripción objetiva de la actividad. Asimismo, se incorporarán observaciones y comentarios técnicos que permitan reflexionar sobre la evolución del proyecto y la toma de decisiones arquitectónicas.

Este documento se constituye como la evidencia formal del compromiso, participación y desempeño técnico de Juan Molinas, sirviendo como instrumento de auditoría para evaluar de forma clara el progreso y la maduración del software durante el período establecido.

Inicio del Proyecto: Arquitectura General de la Solución

El presente proyecto integrador se compone de cuatro infraestructura tecnológicas interconectadas que dan soporte al modelo operativo de la aerolínea:

1. **Arquitectura de Base de Datos (MySQL)**

Diseño de un modelo relacional centralizado compuesto por 13 tablas/entidades principales: Aeropuerto, Avion, Departamento, Servicio, Usuario, Pasajero, Empleado, Asiento, Vuelo, Boleto, Boleto_Servicio, Pago y CheckIn. La lógica de persistencia cuenta con 130 procedimientos almacenados (Stored Procedures) para la manipulación de datos y 130 disparadores (Triggers). Los disparadores se parametrizan predominantemente bajo la condición BEFORE DELETE para garantizar la integridad referencial y actuar como restricciones de seguridad activa, impidiendo fallos de consistencia en cascada al intentar eliminar registros dependientes (por ejemplo, la supresión de un pasajero con boletos activos asociados).

2. **Aplicación de Gestión Interna (Java - Desktop)**

Desarrollo de una aplicación de escritorio orientada a objetos con interfaces gráficas específicas para el personal operativo y los administradores de la empresa. Este entorno interactúa mediante conectividad JDBC con la base de datos centralizada, restringiendo por completo el acceso a usuarios externos o clientes. Implementa módulos CRUD (Create, Read, Update, Delete) completos para las entidades Vuelos, Boletos, Asientos, CheckIns, Pagos, Pasajeros, Empleados y Cuentas. El backend incorpora capas de validación y manejo de excepciones locales que bloquean la inserción o edición de campos con valores incompatibles con las restricciones y tipos de datos nativos de la base de datos.

3. Plataforma Web Comercial y Operativa (HTML, CSS y JavaScript)

Desarrollo de un portal web frontend desacoplado que segmenta la experiencia de usuario en dos perfiles diferenciados: Clientes y Empleados (prescindiendo de un entorno web para el administrador). La interfaz de clientes habilita exclusivamente un flujo de navegación asincrónico para la consulta de itinerarios y la adquisición autónoma de boletos. Por su parte, la vista de empleados integra herramientas CRUD completas para la gestión en tiempo real de Vuelos y Boletos desde cualquier navegador web.

4. Infraestructura y Topología de Redes (Cisco Packet Tracer)

Diseño y simulación de una infraestructura de red corporativa que interconecta un total de 350 terminales de trabajo (PCs) mediante 20 routers distribuidos jerárquicamente por sectores operativos. Cada nodo sectorial cuenta con un router Cisco 4331 enlazado a un switch multicapa Cisco 3650, el cual se conecta a un switch de acceso Cisco 2960. Las terminales (con una densidad variable de entre 6 y 20 computadoras por sector) se vinculan a este último mediante una topología física en estrella para optimizar el rendimiento del canal de comunicación. Los sectores con requerimientos de alta disponibilidad y criticidad operativa incorporan un segundo router Cisco 2960 conectado a subredes redundantes de 6 computadoras con su propio switch de acceso, conformando una topología en malla que asegura la tolerancia a fallos en el transporte de datos.

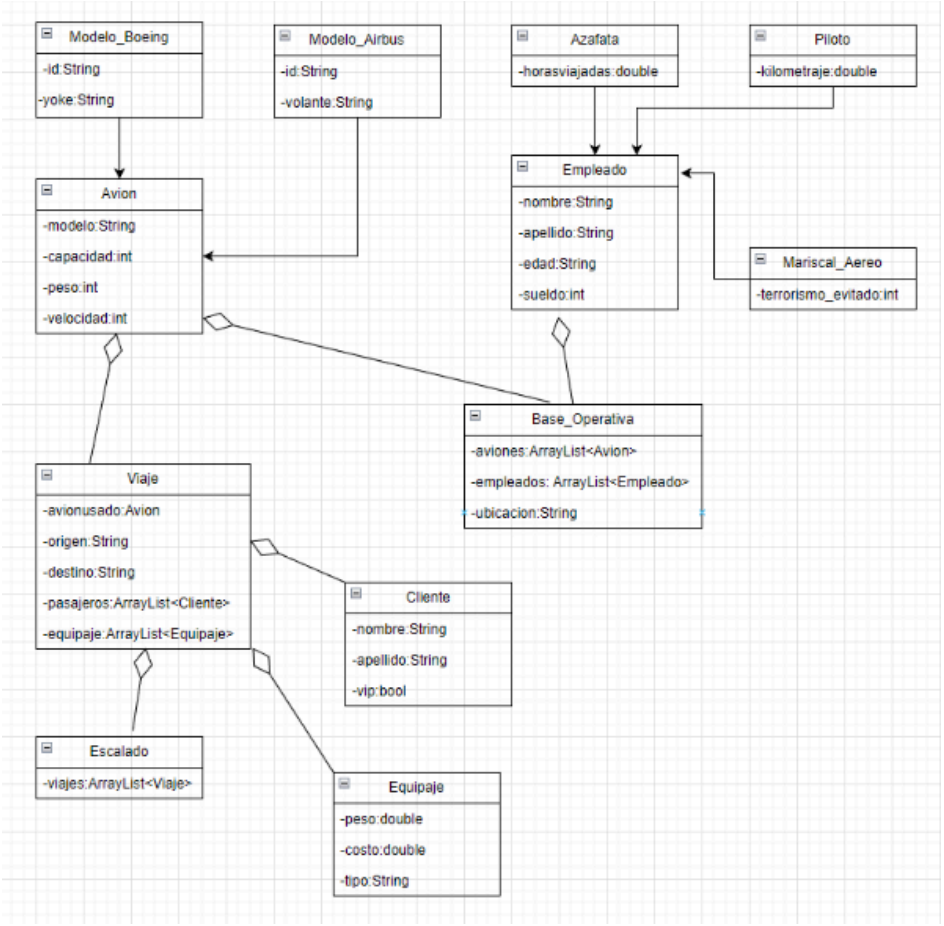
Fecha 30/07/2026

Diseñamos el primer modelo preliminar del diagrama de clases UML para la aplicación de Java. Esta versión inicial se elaboró de forma conceptual antes de contar con las especificaciones técnicas definitivas del alcance del proyecto, por lo que sirvió como base de discusión y fue posteriormente descartada para optimizar la arquitectura del sistema:

En la estructura original, se había planteado una clase padre "Avión" con múltiples clases hijas especializadas. Tras analizar los requerimientos de la base de datos, se resolvió suprimir la herencia y unificar la entidad en una clase individual única llamada "Avion", parametrizando la flota inicial en exactamente cuatro registros de aeronaves dentro del sistema.

De igual manera, descartamos la estructura de herencia para la entidad "Empleado" y sus clases hijas derivadas. En su lugar, implementamos un modelo de asociación mediante la creación de la clase "Departamento". De este modo, los tipos de roles (pilotos, seguridad, administración, etc.) quedan normalizados como atributos de nombre dentro de cada departamento, simplificando el backend.

Finalmente, realizamos una refactorización de términos en el diagrama de clases: la entidad "Viaje" fue reemplazada de forma definitiva por la clase "Vuelo" y se procedió a la adición de nuevas clases complementarias necesarias para cubrir la lógica de negocio del MVP.



Fecha 05/08/2026

Se crea la carpeta de campo grupal en un documento de Google para centralizar el relevamiento de requerimientos del MVP. Diseñamos un cuestionario dividido en 5 campos estratégicos (Piloto, Control de Vuelos, Control Operativo, Encargado de Ganancias y Analista de Costos) para estructurar la lógica del software. Las definiciones del día quedaron distribuidas de la siguiente manera:

En el módulo de Piloto, definimos las preguntas para relevar la duración de viajes, trayectos habituales, personal técnico en cabina por ruta, frecuencia de escalas, el impacto de las tasas aeroportuarias y los costos directos en el aire.

En el módulo de Gestión de Vuelos, establecemos las preguntas sobre la estructura de precios base, asignación de asientos, cobro de equipaje, viabilidad de escalas, algoritmos de rellenado automático, penalizaciones por cancelación de último momento y la verificación legal de habilitaciones para pilotos y azafatas.

En el módulo de Encargado de Ganancias, fijamos los cuestionamientos comerciales sobre el funcionamiento del algoritmo de precios dinámicos, el monitoreo técnico de la competencia y las variables matemáticas para la predicción de ventas futuras.

En el módulo de Control Operativo, estructuramos las preguntas para el diseño de la interfaz de la línea de vuelos activa (retrasos y cancelaciones), la lógica de la calculadora de impacto de demoras (efecto dominó) y el proceso logístico de asignación de aviones a puertas de embarque.

En el módulo de Analista de Costos, cerramos el documento con las preguntas enfocadas en el cálculo de desvíos financieros por factores externos (clima y combustible extra) y la medición de la ganancia real por asiento y kilómetro operativo.

En paralelo, realizamos la creación del repositorio remoto en GitHub para la gestión y el control de versiones distribuido del proyecto. Vinculamos a todos los integrantes del equipo de desarrollo al repositorio central, estableciendo el entorno colaborativo inicial y definiendo las pautas de confirmación de cambios (commits) para la futura integración del código del backend en Java, la interfaz web y los scripts de la base de datos MySQL.

Fecha 06/08/2026

Procesamos las respuestas del cuestionario de relevamiento y realizamos la bajada técnica de los requerimientos funcionales para el MVP, traduciendo la información del modelo operativo low-cost en las reglas de negocio que estructurarán el backend.

Para el módulo de Piloto, establecemos las limitaciones lógicas del sistema de asignación de rutas, definiendo un bloqueo automático para tramos mayores a 5 horas para forzar patrones en espejo. También configuramos los parámetros lógicos para las alertas visuales en rampa que controlarán el turnaround crítico de 30 minutos y los tiempos de descanso de la tripulación técnica.

En Gestión de Vuelos, fijamos la lógica de backend para la estrategia de unbundling (cobro desglosado de equipaje y servicios adicionales) y el diseño del algoritmo de dispersión intencional de asientos en el check-in gratuito. Asimismo, parametrizamos las reglas ultra-estrictas del Crew Management System para el bloqueo automatizado de tripulaciones por límites de horas legales y la reasignación inmediata de no-shows para personal en traslado (deadheading).

Para el Encargado de Ganancias, definimos la estructura matemática del motor de precios dinámicos según las curvas de reserva históricas para forzar un factor de ocupación del 90%. Adicionalmente, establecemos los requisitos técnicos de latencia, frecuencia y brecha de precios para el submódulo de inteligencia competitiva basado en web scraping.

En Control Operativo, determinamos el diseño de la interfaz gráfica del OCC basada en diagramas de Gantt de alta densidad alimentados por telemetría ACARS para actualizar retrasos en tiempo real. También estructuramos las variables de las calculadoras de impacto financiero (multas de slots, combustible, compensaciones) y la simulación del efecto dominó para predecir colapsos de jornadas (Duty Period Time Out).

Finalmente, para el Analista de Costos, estructuramos las fórmulas para las auditorías automatizadas de desvíos económicos en el aire (Actual vs Plan), aislando el combustible cargado por discrecionalidad del comandante. Cerramos definiendo la lógica de reportes ejecutivos para el cálculo automatizado de las densidades económicas RASK (incluyendo Ancillary RASK) y CASK.

Fecha 07/08/2026

Dedicamos la jornada a listar y estructurar la totalidad de las actividades pendientes del proyecto para establecer las bases de la planificación temporal. Suspendimos momentáneamente el desarrollo técnico para realizar el desglose de tareas y categorizarlas en cinco fases críticas, definiendo el alcance global del desarrollo:

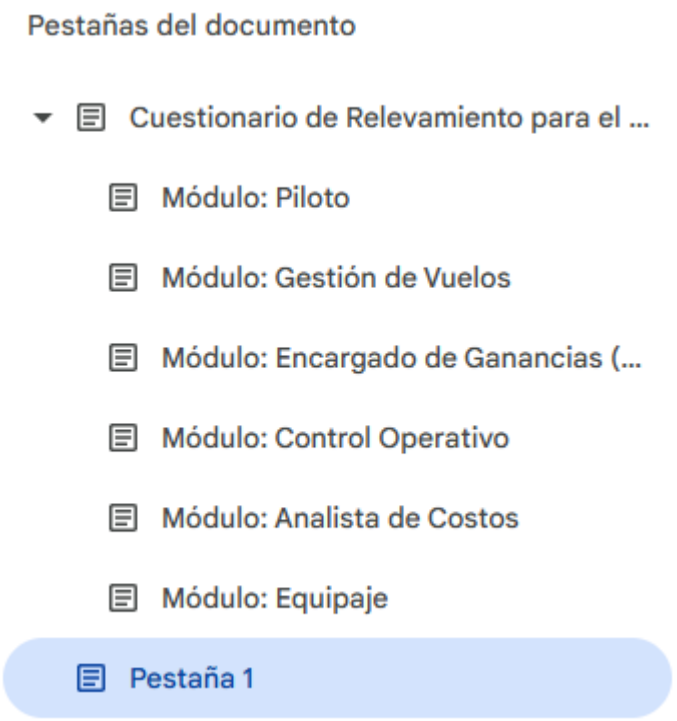
En la fase de Análisis, definimos las actividades de búsqueda de información (A1), el relevamiento final (A2), la creación del modelo Gantt, PERT, FODA y la metodología Scrum (A3), el listado de requisitos funcionales y no funcionales fijando un objetivo de 25 de cada uno (A4), la documentación técnica general (A5) y el cálculo matemático del Mínimo Producto Viable (A6).

En la fase de Diseño, establecemos las tareas de creación física de la base de datos (B1), el diseño detallado de la Interfaz de Usuario UI (B2), el diseño del logo corporativo de la empresa (B3), el armado del diagrama UML para programación orientada a objetos (B4) y la redacción del documento formal de presentación de la problemática junto a su solución (B5).

En la fase de Desarrollo, mapeamos los módulos que se programarán en la aplicación de Java y la web, los cuales incluyen la gestión de vuelos (C1), gestión de pasajeros (C2), gestión de ventas de boletos y precios (C3), gestión de equipaje (C4), creación de manuales técnicos (C5), servicio de atención al cliente (C6), el módulo de check-in (C7), el panel de administración (C8) y la actualización continua de la carpeta de campo grupal (C9).

En las fases de Pruebas e Implementación, fijamos la actividad de búsqueda y depuración de errores del código (D1), el despliegue e instalación final del sistema completo, la entrega de manuales y la presentación final del proyecto (E1).

Finalmente, procesamos los requerimientos específicos del profesor para el diseño de la topología en Cisco Packet Tracer. Se registraron las densidades de red obligatorias por sector para el armado de la infraestructura de las aerolíneas: Check-in (10 PCs), Atención al Cliente (6 PCs), Operaciones (8 PCs), Equipaje (5 PCs) y Administración (5 PCs).
Ademas de todo eso se crearon pestañas en el documento para organizarlo de mejor manera y facilitar la busqueda del material de trabajo porque las respuestas del cuestionario ocupaban demasiado espacio.



Fecha 10/08/2026

Se trabajó sobre el registro de campo del proyecto, específicamente en la normalización y depuración del transcrito de la entrevista realizada por el compañero Joshua al piloto Octavio. El objetivo de la jornada fue corregir los errores de formato del documento original y adaptar las respuestas técnicas que presentaban inconsistencias de transcripción, asegurando la fidelidad de los datos relevados para el módulo de Piloto.

A continuación, se adjunta el segmento corregido y validado de la entrevista:

Entrevista de Relevamiento
Entrevistado: Octavio (Piloto Comercial)

¿Cuál es la duración del viaje y el trayecto más largo que realizan habitualmente?
En la aviación comercial realizamos principalmente trayectos regionales o de cabotaje, lo que implica hacer múltiples vuelos cortos en una misma jornada operativa y evitar por completo los cruceros oceánicos; actualmente, mi tramo más largo es la ruta entre Londres y Larnaca, en Chipre, que toma unas 5 horas de vuelo entre ida y vuelta.

¿Cuántos pilotos y personal técnico se necesitan en cabina según el tipo de ruta establecida?
Hoy en día se necesitan exactamente dos personas en la cabina de mando, cumpliendo los roles de piloto y copiloto, ya que la figura del técnico de vuelo desapareció por la modernización tecnológica, aunque si la ruta es de larga distancia el sistema exige sumar más pilotos para los turnos de relevo y descanso.

¿Cómo se maneja la cantidad y frecuencia de las escalas en la operación diaria?
La cantidad y frecuencia de las escalas se maneja puramente según la demanda de pasajeros de cada destino, operando múltiples frecuencias diarias en ciudades grandes como Buenos Aires o Córdoba, mientras que en lugares con menos movimiento como Río Grande o destinos turísticos como Bariloche la frecuencia se adapta según la temporada.

¿Qué impacto operativo tienen los diferentes precios y tasas de cada aeropuerto en la ruta?
Sobre el impacto operativo que tienen los diferentes precios y tasas de cada aeropuerto en la ruta no te sabría decir, ya que no poseo información sobre ese tema en este momento.

¿Cuáles son los principales costos directos asociados exclusivamente a la operación en el aire?
Los principales costos directos de la operación aérea cuando el avión ya está en el aire son el combustible especial que utilizamos, conocido como Jet A-1 (kerosén de aviación), y los gastos

Fecha 11/08/2026

Tomamos como estructura de partida el diagrama inicial desarrollado por el compañero Joaquín Ferramola para comenzar la planificación temporal y operativa del proyecto. El trabajo de la jornada se centró en la integración y asignación de las actividades de Análisis, Diseño, Desarrollo, Pruebas e Implementación que habíamos desglosado previamente.

Utilizando las herramientas de Google Docs y Google Sheets de forma colaborativa, volcamos la totalidad del listado de tareas en una matriz de seguimiento. Durante la sesión, realizamos la estimación de tiempos de ejecución para cada bloque técnico y procedimos con la asignación formal de responsables y encargados para cada actividad, dejando consolidada la estructura analítica antes de la generación definitiva del diagrama de Gantt.

Tarea	Símbolo	Dependencias	Participante	Duración (Días)	Estado	3
Recoleccion de informacion	A0	NINGUNO	Joaquin Ferramola, Juan Molinas, Jeremias Benitez, Joshua Chodoz, Gonzalo Delgado	5	Completada	
Entrevista	A1	A0	Joshua Chodoz	1	Completada	
Pasar la entrevista a limpio	A2	A1	Juan Molinas	1	Completada	
Relevamiento	A3	A1	Joaquin Ferramola, Juan Molinas, Jeremias Benitez, Joshua Chodoz, Gonzalo Delgado	1	Completada	
Análisis foda	A5	A3	Gonzalo Delgado	1	Completada	
Modelo gantt	A4	A3	Joaquin Ferramola, Gonzalo Delgado, Juan Molinas	2	Completada	
Requisitos funcionales	A8	A3	Jeremias Benitez	2	No iniciada	
Modelo pert	A6	A4	Joaquin Ferramola	1	No iniciada	
Modelo scrum	A7	A4	Juan Molinas	1	No iniciada	
Requisitos no funcionales	A9	A8	Joshua Chodoz	2	No iniciada	
Documentacion	A10	A8, A9	Juan Molinas	3	No iniciada	
Calcular el minimo producto viable	A11	A10	Gonzalo Delgado	1	Bloqueada	
DER	B0	A11		1	No iniciada	
Creacion de la base de datos	B1	B0		3	No iniciada	

Se puede ver en la imagen que al ser el primer modelo del Gantt todas las actividades no estaban asignadas a sus responsables

Fecha 13/08/2026

Realizamos la asignación formal y distribución de responsabilidades para la totalidad de las tareas que se habían estructurado al inicio del proyecto. Vinculamos de manera definitiva a cada integrante del equipo con sus respectivos bloques de desarrollo en la base de datos, la aplicación de Java, la interfaz web y el diseño de la infraestructura de red.

Cabe destacar que esta planificación responde al alcance inicial relevado, quedando registrado en la bitácora que algunas de estas actividades sufrieron modificaciones,

reemplazos o fueron dadas de baja en etapas posteriores del desarrollo para adecuarse a la evolución técnica del sistema.

Fecha 18/08/2026

Diseñamos y consolidamos la carátula formal para la carpeta de campo grupal. En esta sesión de documentación, unificamos la identidad institucional del proyecto, estructurando los datos de identificación de los integrantes del equipo, el título oficial del software, los módulos de desarrollo y el marco académico correspondiente, dejando el documento formalmente presentado para las futuras revisiones.

Proyecto Interareal

Temática: Aerolineas Low-Cost (LCC)

Integrantes:

- Juan Molinas (L)
- Joaquín Ferramola
- Gonzalo Delgado
- Jeremías Benitez
- Joshua Chodoz

Índice:

1 Cuestionario de Relevamiento
  1.1 Entrevista
  1.2 Módulo: Piloto
  1.3 Módulo: Gestión de Vuelos
  1.4 Módulo: Encargado de Ganancias (Revenue Manager)
  1.5 Módulo: Control Operativo
  1.6 Módulo: Analista de Costos
  1.7 Módulo: Equipaje
2 MVP (Minimum Viable Product)
3 Que soluciona nuestro programa
4 Java
  4.1 UML
5 Redes
  5.1 Presupuesto
  5.2 Topografía

Fecha 19/08/2026

Llevamos a cabo una sesión de trabajo crítico en la que se redefinió y acotó formalmente el alcance del proyecto para adaptarlo a los tiempos reales de desarrollo. En esta jornada, se establecieron las directrices definitivas del software y se consolidó la documentación técnica obligatoria:

Procedimos a la depuración de las actividades planificadas al inicio del proyecto. Optamos por simplificar el sistema descartando los módulos que no aportaban valor inmediato al núcleo operativo, dejando exclusivamente las verticales de Gestión de Vuelos y Gestión de Equipaje como ejes centrales de la aplicación.

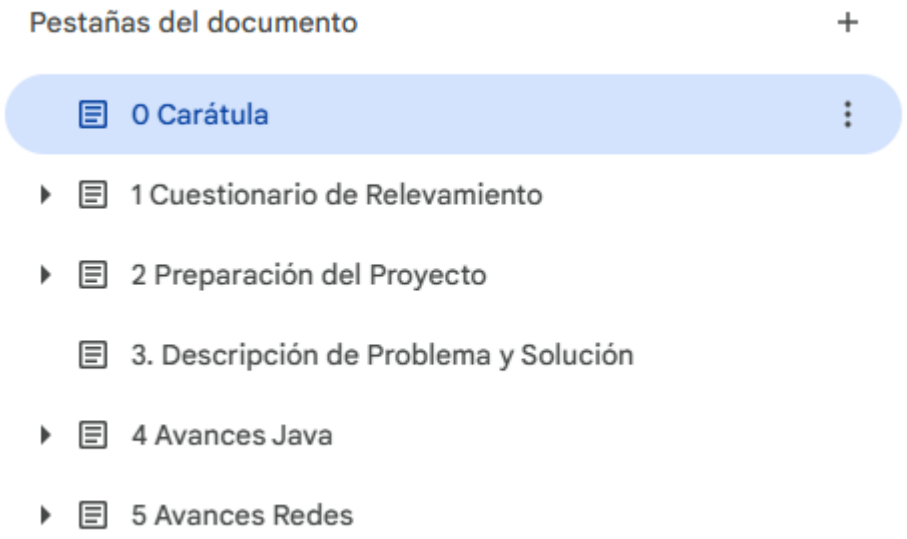
Diseñamos conceptualmente la arquitectura y los accesos para el MVP de la aerolínea Chorifly, estructurando un sistema de cuentas centralizado con tres niveles de permisos: pasajeros, empleados y administradores. Definimos el flujo para que los administradores realicen la carga de vuelos e itinerarios desde la aplicación de Java, permitiendo que estos impacten de forma automática en el calendario de la plataforma web. Para los empleados, se estructuró un panel web y local enfocado en el registro manual de pasajeros y la asignación de asientos (manual o automatizada). Para los pasajeros, se delineó el buscador dinámico en la web orientado a la compra autónoma con cálculo de peso de equipaje y selección de servicios complementarios.

Elaboramos el documento formal que justifica el desarrollo del software. Diagnosticamos el estado actual de la empresa, caracterizado por la ausencia de plataformas tecnológicas, procesos manuales en papel y archivos de Excel aislados que derivaban en errores operativos y demoras aeroportuarias. Acto seguido, redactamos la propuesta de solución basada en los dos entornos de software complementarios y descentralizados que aseguran la continuidad del negocio y el control operativo absoluto.

Finalizamos la jornada redactando y aprobando la lista definitiva de ingeniería de requerimientos. Logramos consolidar un total de 25 requisitos funcionales (donde destacan el control de ventanas para el check-in, la lógica de asignación aleatoria de asientos y el cálculo de niveles de equipaje) y 25 requisitos no funcionales (enfocados en garantizar un uptime web del 99.9%, tolerancia a la concurrencia, transacciones ACID para evitar la duplicidad de asientos y un consumo de memoria RAM optimizado para la aplicación en Java).

Se procedió a la reestructuración y limpieza profunda del borrador de la carpeta de campo grupal. Volcamos de manera sistemática toda la información técnica validada hasta la fecha y eliminamos de forma definitiva las secciones correspondientes a los módulos descartados para evitar redundancias y asegurar el avance limpio del proyecto.

A continuación, se detalla la disposición visual de las pestañas organizativas del borrador técnico de trabajo:



Fecha 21/08/2026

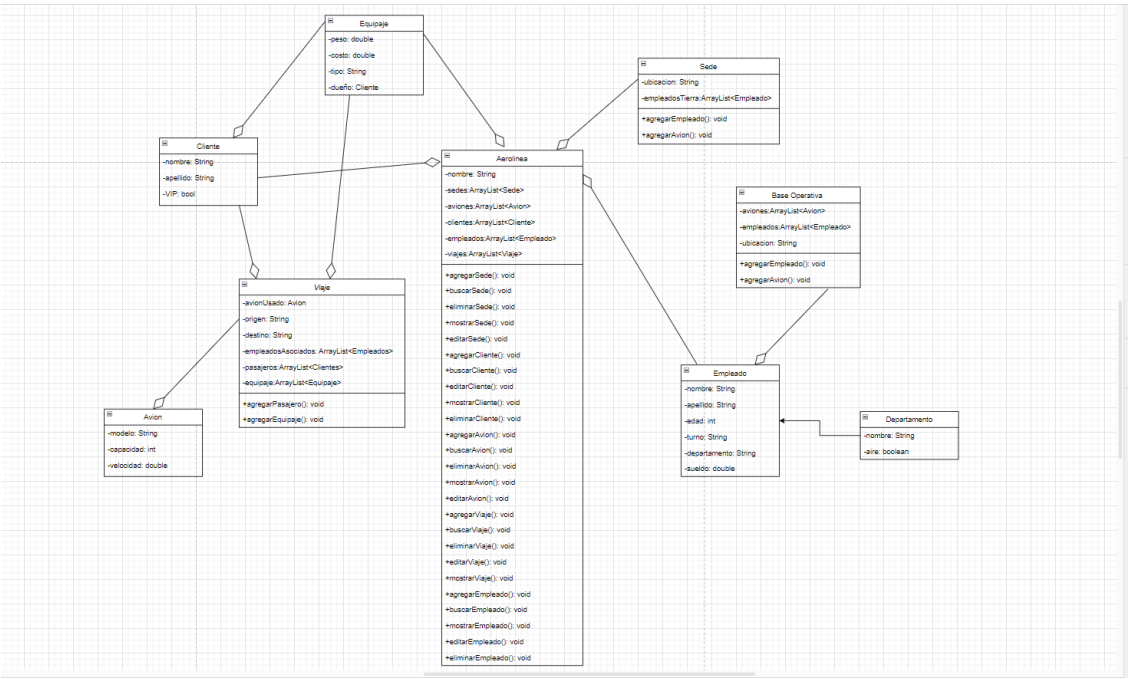
Llevamos a cabo una reingeniería de la planificación temporal del proyecto. Tras evaluar la primera versión del diagrama de Gantt, resolvimos descartar el documento por presentar complejidades en su mantenimiento manual y falta de flexibilidad para reflejar de forma ágil las modificaciones de las tareas.

En su lugar, procedimos a estructurar un nuevo cronograma utilizando la plantilla predeterminada de Diagrama de Gantt en formato de hoja de cálculo. Volcamos la totalidad de las actividades de Análisis, Diseño, Desarrollo, Pruebas e Implementación que habían sido aprobadas previamente para el MVP, aprovechando las funciones de automatización de la plantilla para calcular las barras de duración en días, los porcentajes de progreso por bloque y la visualización de la ruta crítica del desarrollo.

Fecha 24/08/2026

Se realizó el segundo modelo UML del proyecto, representando las clases Aerolinea, Sede, Base Operativa, Avion, Viaje, Cliente, Equipaje, Empleado y Departamento, junto con sus respectivas relaciones. La clase Aerolinea se conecta con las sedes, los aviones, los clientes, los empleados y los viajes, mientras que Viaje se relaciona con Avion, Cliente, Empleado y Equipaje. También se incorporaron las relaciones entre Sede y Base Operativa con los empleados y aviones, y entre Empleado y Departamento.

Además, se crearon los archivos correspondientes en GitHub, incluyendo los enlaces a las carpetas de campo individuales y a la grupal.



Fecha 31/08/2026

Se creó la base de datos del proyecto utilizando como referencia el modelo proporcionado por un compañero. Esta base de datos fue modificada posteriormente para mejorar su estructura, incorporar nuevos datos y establecer relaciones entre sus tablas.

La primera versión estaba compuesta por las tablas Aeropuerto, Avion, Departamento, Pasajero, Servicio, Usuario, Asiento, Empleado, Vuelo, Boleto, Boleto_Servicio, Pago y CheckIn.

La tabla Aeropuerto almacena los datos de los aeropuertos, incluyendo su nombre, país y ciudad. La tabla Avion registra los aviones disponibles, sus modelos y capacidades. Por otro lado, Departamento organiza las áreas de trabajo de la aerolínea, mientras que Servicio almacena los servicios adicionales que pueden contratar los pasajeros, junto con sus precios y descripciones.

La tabla Usuario administra las cuentas de acceso al sistema y distingue a los usuarios según sus privilegios. La tabla Pasajero almacena la información personal de los clientes que utilizan la aerolínea, mientras que Empleado registra los datos de los trabajadores y los vincula con sus respectivos departamentos.

La tabla Asiento representa los asientos disponibles en cada avión y permite registrar su disponibilidad. La tabla Vuelo almacena los vuelos programados, relacionándolos con el avión utilizado y los aeropuertos de origen y destino, además de registrar sus horarios y estados.

La tabla Boleto vincula a cada pasajero con un vuelo y un asiento, registrando también la fecha y el precio base. La tabla intermedia Boleto_Servicio permite asociar varios servicios adicionales a un mismo boleto. De esta manera, un pasajero puede contratar diferentes servicios sin necesidad de crear una tabla independiente para cada combinación.

La tabla Pago registra los pagos correspondientes a los boletos, incluyendo los costos de los servicios y el importe total. Finalmente, CheckIn permite registrar la información del proceso de registro previo al embarque, relacionando los vuelos y los pagos con la puerta de embarque correspondiente.

Las relaciones entre las tablas se establecen mediante claves primarias y claves foráneas. Estas permiten identificar los registros y vincularlos entre sí, evitando que se asignen, por ejemplo, boletos a pasajeros o vuelos que no existan en la base de datos.

En conjunto, la base de datos permite organizar la información de la aerolínea, desde la administración de usuarios, empleados, aviones y vuelos hasta la venta de boletos, la contratación de servicios adicionales, el registro de pagos y el proceso de check-in.

Durante el desarrollo del proyecto, esta estructura se fue actualizando hasta alcanzar una versión final con campos adicionales, nuevas restricciones y relaciones más completas entre las tablas.

También se subieron a la carpeta de campo grupal los registros correspondientes al diagrama de Gantt, el diagrama PERT y el análisis FODA. Se creó el primer archivo `.java`, dando inicio al desarrollo de la aplicación.

Por último, se investigaron y definieron los materiales que se utilizarían para construir la red en Cisco Packet Tracer. Se seleccionaron los routers ISR 4331, los switches centrales Catalyst 3650-24PS, los switches de acceso Catalyst 2960 y el servidor Server-PT.

Se comenzó a diseñar la topología de red en Cisco Packet Tracer, pero debido a un problema con la máquina se perdió el diseño realizado, aunque la idea general ya estaba planteada. Se definió el uso de routers ISR 4331, switches Catalyst 3650 como nodos centrales de cada red o sector, switches Catalyst 2960 como dispositivos secundarios y servidores Server-PT.

También se definió la distribución de las computadoras para las 20 áreas de la empresa. En seis sectores considerados especialmente críticos se planificaron seis computadoras adicionales para implementar una red en malla, con el objetivo de garantizar una mayor tolerancia a fallas. La importancia de estos sectores requería que, ante la posible falla de un equipo o una conexión, la red pudiera seguir funcionando mediante rutas alternativas, evitando que un único problema provocara la caída de todo el sistema del sector. Los seis sectores seleccionados fueron Dirección de Operaciones, Mantenimiento de Aeronaves (MRO), Despacho de Vuelos, IT y Sistemas Críticos, Control de Tráfico Aéreo (Enlace) y Gestión de Crisis. La distribución inicial contemplaba 343 computadoras, a las que se sumarían las 36 adicionales para estas redes en malla, alcanzando un total previsto de 379 computadoras. La maqueta virtual debía contar con el 100 % de los componentes planificados.

Además, comencé a desarrollar el primer diseño de la página web de Chorifly utilizando HTML y CSS, sin incorporar JavaScript por el momento. Se diseñaron elementos iniciales como el encabezado, el menú de navegación, el calendario, la sección principal y el pie de página. Sin embargo, el desarrollo web quedó en espera, ya que pasé a dedicarme principalmente a la base de datos y a la aplicación en Java.

Por último, en GitHub se crearon las branches correspondientes a cada materia y se comenzó a organizar el repositorio con mayor precisión, debido al creciente número de archivos incorporados al proyecto.

Fecha 02/09/2026

Se realizaron modificaciones en la base de datos original. Se agregó el campo aeropuerto_id a la tabla de empleados para identificar el aeropuerto al que pertenece cada uno. También se incorporó usuario_id en las tablas de pasajeros y empleados, permitiendo vincularlos con un usuario del sistema. Esta relación admite valores nulos, pero cada pasajero o empleado puede estar asociado, como máximo, a un único usuario.

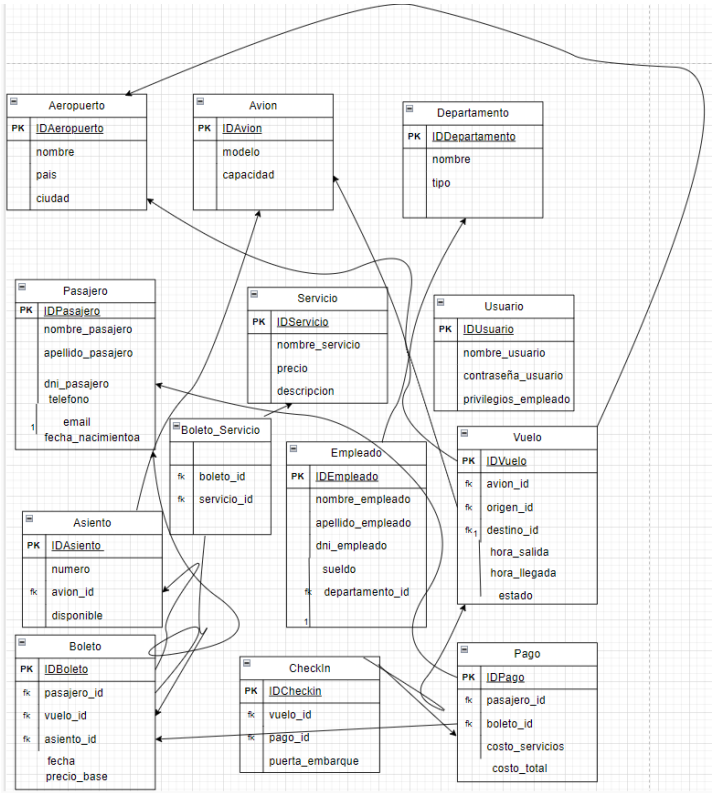
Se subió a GitHub una primera versión de la topología de red realizada en Cisco Packet Tracer, aunque todavía se encontraba incompleta.

Para mejorar la organización del repositorio, se comenzaron a crear archivos README en todas las branches, con el objetivo de documentar su contenido y conocer mejor la distribución de los archivos correspondientes a cada materia.

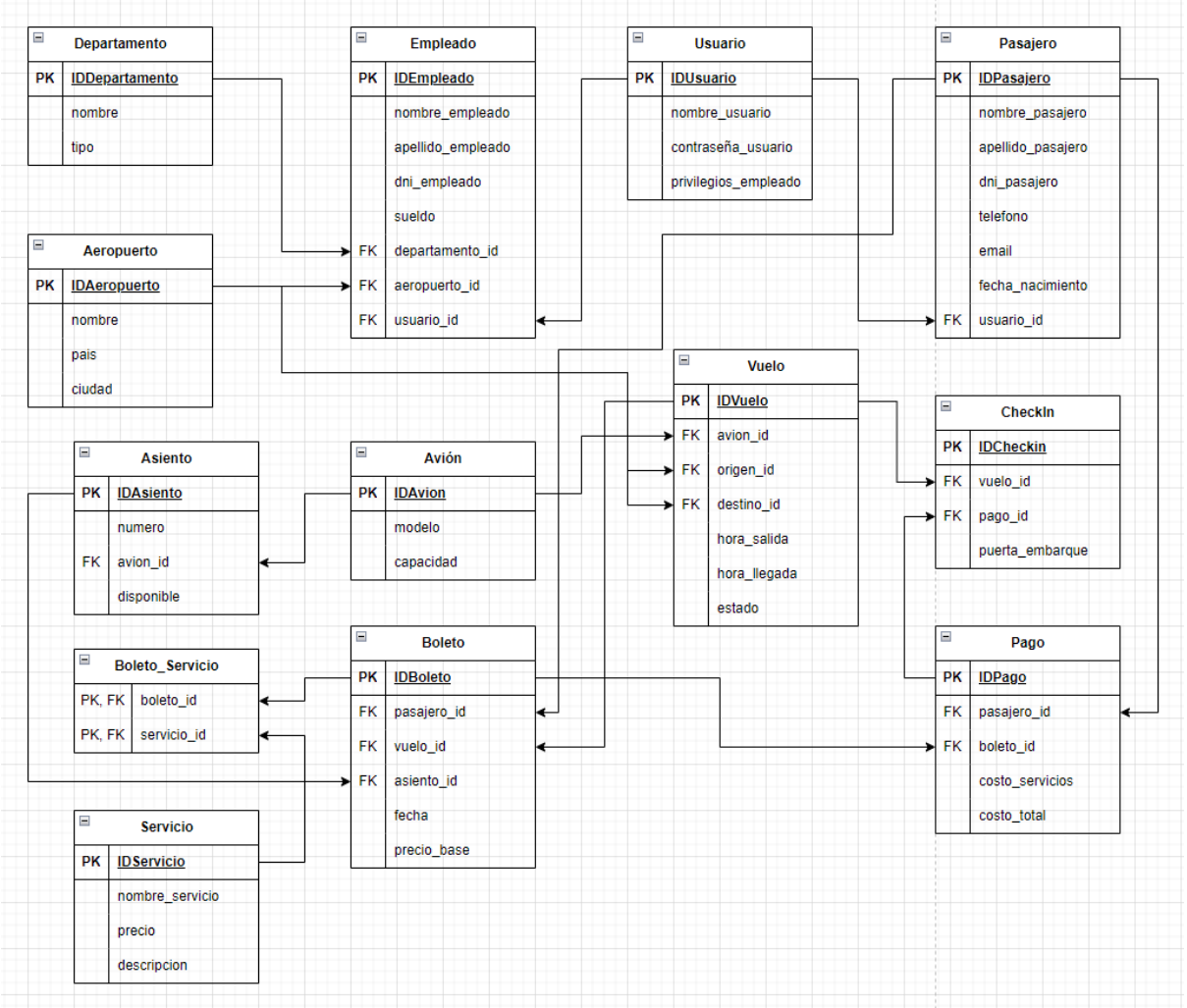
Además, se creó un archivo que contiene un enlace a un documento de Google con el CRUD inicial del proyecto. En un primer momento, Gonzalo Delgado comenzó a trabajar en esta parte, pero posteriormente pasó a encargarse del desarrollo de la página web. A partir de ese momento, comencé a desarrollar desde cero la aplicación en Java, con el objetivo de conectarla y sincronizarla con la base de datos del sistema.

Además, se revisó y corrigió el DER (Diagrama Entidad-Relación) enviado por el compañero Joshua Chodos, ya que presentaba numerosos errores de prolijidad y organización. Debido a estos inconvenientes, fue necesario rehacer el diagrama desde cero para obtener una estructura más clara y ordenada, que representara correctamente las entidades y sus relaciones y sirviera como base para el desarrollo de la base de datos del proyecto.

(Diseño por Joshua Chodos)



(Diseño por Juan Molinas)



Cabe aclarar que esta no es la versión final pero la versión final solo tiene corregidos un par de detalles en los nombres de algunos campos. Por ejemplo: Tabla Asiento: *numero* fue reemplazado con Tabla Asiento: *codigo*.

Fecha 07/09/2026

Se crearon los archivos README.md de la branch principal (main) y de las distintas branches del repositorio de GitHub, utilizando el formato Markdown para organizar y presentar la información de manera clara.

En el README principal se estableció una estructura con títulos y subtítulos para identificar la temática del proyecto, los integrantes del equipo, las materias involucradas, la descripción general de Chorifly, los objetivos principales y el registro de avances. También se incorporaron enlaces a las distintas branches, permitiendo acceder directamente al contenido correspondiente a cada asignatura.

Por otro lado, los README de las branches específicas siguieron un formato similar, adaptado a las necesidades de cada materia. Se utilizaron encabezados para identificar la asignatura y su propósito, listas para enumerar los archivos disponibles y breves descripciones para explicar el contenido de cada uno. Además, se agregaron enlaces para navegar entre las ramas y regresar a la principal. En el caso de Análisis de Sistemas, por ejemplo, se documentaron los archivos relacionados con el FODA, los diagramas de Gantt y PERT, el MVP, los requisitos, Scrum y los distintos análisis del proyecto.

Esta forma de organización permitió centralizar la información de referencia de cada rama, facilitar la navegación por el repositorio y mantener una estructura uniforme a medida que aumentaba la cantidad de archivos. Durante esta jornada, el trabajo se concentró en la documentación y organización de GitHub, sin realizar otros avances en el desarrollo del sistema.

Fecha 15/09/2026

En primer lugar, se crearon los INSERT necesarios para cargar los datos iniciales de la base de datos y disponer de un escenario de pruebas más completo. Se incorporaron registros para las tablas de departamentos, aeropuertos, aviones, servicios, usuarios, empleados y pasajeros, además de datos relacionados con asientos, vuelos, boletos, servicios contratados, pagos y check-in. Entre estos registros se incluyeron algunos datos ficticios con nombres elegidos de manera personal, para darle una identidad propia al proyecto. La intención era contar con información suficiente para comprobar las relaciones entre las distintas tablas y comenzar a probar las funcionalidades del sistema sin tener que cargar los datos manualmente cada vez. Sin embargo, se trataba de una carga inicial que podía modificarse a medida que avanzara el desarrollo, por lo que posteriormente algunos registros serían eliminados o reemplazados al determinarse que no eran esenciales para el funcionamiento del programa.

Por otra parte, se comenzó a trabajar específicamente en la distribución de los asientos de los aviones. Para ello, se creó un primer archivo SQL que permitía insertar los asientos correspondientes a una única aeronave. La distribución inicial contemplaba seis asientos por fila, identificados mediante las letras de la A a la F, y distinguía las ubicaciones de ventana, medio y pasillo. Cada asiento contaba con un identificador propio, un código, una referencia al avión al que pertenecía, un tipo y un estado de disponibilidad. En esta primera versión se generaron 31 filas de seis asientos, alcanzando un total de 186 lugares para un avión. Todos se cargaron inicialmente como disponibles, con el propósito de utilizarlos en las primeras pruebas de selección y representación gráfica desde Java.

Al analizar cómo se integraría este modelo con la aplicación, se detectó una dificultad importante relacionada con la manera en que se vinculaban los asientos y los vuelos. Como los asientos estaban asociados directamente al avión y no a un vuelo específico, permitir que una misma aeronave tuviera varios vuelos programados simultáneamente podía generar conflictos en la disponibilidad de los lugares. Por ejemplo, dos vuelos asociados al mismo avión consultarían el mismo conjunto de asientos, lo que dificultaría determinar correctamente cuáles estaban disponibles para cada operación. Para evitar esta situación, se decidió establecer como condición que cada avión pudiera tener, como máximo, un vuelo en estado Programado al mismo tiempo. Posteriormente, se incorporó este estado al conjunto de estados posibles de los vuelos, de manera que la programación respetara la lógica definida para la asignación de asientos.

A partir de estas revisiones, también se decidió reducir la cantidad de asientos por avión de 186 a 180. Este cambio buscaba facilitar la representación gráfica de los lugares en Java, ya que trabajar con una cantidad divisible en 30 filas de seis asientos simplificaba la distribución visual de los botones y la organización de la ventana de selección. De esta manera, el diseño se adaptaba mejor a las necesidades de la interfaz que se pretendía desarrollar.

Junto con esta modificación, se revisó la forma de identificar cada asiento. Inicialmente se utilizaban códigos como 1A, 1B o 1C, donde el número representaba la fila y la letra indicaba la posición. Sin embargo, se decidió pasar a un formato de dos cifras para mantener una nomenclatura uniforme, por lo que un código como 1A pasó a representarse como 01A. Para implementar este cambio, la columna numero, que originalmente era de tipo entero (INT), fue reemplazada por una columna codigo de tipo texto (VARCHAR), capaz de almacenar la identificación completa del asiento en un único campo. Así, el formato evolucionó hacia

códigos como 01A, 02B o 30F, facilitando su lectura, búsqueda y posterior utilización desde la aplicación Java.

En conjunto, estos avances permitieron no solo preparar los datos necesarios para las primeras pruebas, sino también revisar decisiones de diseño que afectaban directamente al funcionamiento del sistema. La carga inicial proporcionó registros para comprobar las relaciones entre las entidades, mientras que el desarrollo del modelo de asientos permitió anticipar las necesidades de la interfaz gráfica y detectar restricciones que debían resolverse antes de completar su implementación. De esta manera, la jornada representó un avance importante tanto en la preparación de la base de datos como en la definición de la lógica que conectaría esta estructura con la aplicación Java.

Fecha 16/09/2026

Durante esta jornada se continuó trabajando sobre la base de datos de Chorifly. Luego de haber preparado e incorporado los INSERT necesarios para cargar los datos iniciales y comprobar la estructura de las tablas y sus relaciones, se realizó una limpieza de los datos de prueba. Tras revisar los registros que se habían preparado, se decidió eliminar los INSERT correspondientes a tablas que ya no se consideraban necesarias para el desarrollo actual del proyecto. El objetivo fue evitar la acumulación de información innecesaria y conservar únicamente los datos que resultaran útiles para las pruebas y el funcionamiento de la aplicación. De esta manera, se ajustó la base de datos a las necesidades reales del proyecto y se dejó pendiente para etapas posteriores la creación de vistas, triggers y procedimientos almacenados.

También se completó la organización del repositorio de GitHub, incluyendo las branches correspondientes a cada materia, sus archivos dedicados y los respectivos README.md. Esto permitió mantener una estructura ordenada y facilitar el acceso a los materiales y documentos del proyecto.

En el área de Redes, junto con Joaquín Ferramola, se terminó el diseño de las 20 subredes en Cisco Packet Tracer y se configuraron los cuatro servidores previstos: DHCP, DNS, HTTP y FTP. Sin embargo, todavía quedaba pendiente conectar todas las subredes entre sí. Para avanzar con esta etapa, se planificó incorporar módulos NIM-2T a los routers Cisco ISR 4331, con el propósito de disponer de interfaces seriales y realizar el enrutamiento estático entre las distintas redes.

Por otra parte, junto con Gonzalo Delgado, se comenzaron a diseñar las primeras ventanas de la aplicación Java utilizando JFrame. Durante esta etapa también se empezó a definir dónde y cómo se implementarían las verificaciones y validaciones del sistema.

Finalmente, el diseño inicial de las ventanas fue descartado y reemplazado por un nuevo modelo más ordenado y visualmente prolijo. Esta nueva propuesta conservaba los mismos campos y funcionalidades previstos, pero incorporaba una mejor presentación y el uso de colores para facilitar la identificación de los distintos elementos de la interfaz.

Fecha 18/09/2026

Durante esta jornada se realizó una modificación importante en el MVP del proyecto Chorifly. Se decidió cambiar la forma en que se manejaría el equipaje adicional, eliminando la posibilidad de calcular su costo según el peso. En su lugar, el precio del equipaje extra pasaría a establecerse mediante los servicios disponibles dentro del sistema. De esta manera, si un pasajero no contrata el servicio de equipaje extra, no podrá llevar equipaje adicional. Esta decisión permitió simplificar la lógica que se deberá programar, evitando tener que calcular precios según diferentes pesos y centralizando el costo directamente en el servicio correspondiente. Por lo tanto, el equipaje extra dejaría de manejarse como una característica independiente y solamente estaría disponible a través de la contratación del servicio.

En cuanto a la base de datos, se realizaron varias modificaciones para corregir omisiones y mejorar la validación de los datos. Se agregó el campo email a la tabla Usuario, ya que era necesario almacenar el correo electrónico de cada usuario y se había omitido por descuido en el diseño inicial. Como consecuencia de este cambio, se actualizaron los INSERT correspondientes para que contemplaran el nuevo campo y coincidieran con la estructura actualizada de la tabla.

También se modificó el tipo de dato de la columna correspondiente a la fecha de nacimiento, pasando de DATETIME a DATE, dado que para representar una fecha de nacimiento no era necesario almacenar una hora específica. Además, se incorporó la restricción NOT NULL en la mayoría de las columnas de la base de datos, con el objetivo de impedir que se registraran valores nulos en campos que debían contener información obligatoria. De esta manera, si desde Java se intentaba guardar información incorrecta o incompleta, la base de datos podría

generar una excepción que permitiera identificar el problema mediante el manejo de SQLException y la consulta del error correspondiente. Esto buscaba facilitar la detección y corrección de errores durante el desarrollo del programa.

Por otro lado, se agregó un nuevo departamento relacionado con el equipaje. Este se incorporó como otra opción dentro de los departamentos disponibles, sin que fuera necesario implementar una funcionalidad específica adicional en esta etapa del proyecto.
También se modificaron los nombres de las columnas de la tabla Vuelo: los campos hora_salida y hora_llegada pasaron a llamarse fecha_hora_salida y fecha_hora_llegada, respectivamente. El cambio buscaba representar con mayor claridad que ambos campos almacenaban tanto la fecha como la hora de cada vuelo.

Por otra parte, se comenzó a desarrollar la estructura base del programa en Java utilizando el IDE NetBeans, organizando el código bajo el paquete "chorifly". El objetivo de esta etapa fue crear el mapeo de clases necesarias para representar los elementos principales del sistema y preparar la estructura del modelo de datos orientada a objetos (POJO), para posteriormente entregarle esta base a Gonzalo Delgado, quien continuará con el desarrollo de las vistas y las interfaces gráficas.

Las primeras clases de entidades creadas en el código fuente fueron Aeropuerto, Avion, Asiento, Departamento, Persona, Empleado, Pasajero y Usuario, además de la clase controladora principal denominada Aerolinea y la clase ejecutable Chorifly. La idea fue representar fielmente las entidades de la base de datos mediante objetos en Java, de manera que, por ejemplo, un pasajero pudiera existir como una instancia de la clase Pasajero y utilizarse posteriormente para realizar operaciones sobre la persistencia. En esta primera

etapa, las clases se centraron exclusivamente en la representación de los datos y encapsulamiento, sin incorporar todavía la lógica de las operaciones CRUD ni patrones DAO. Para organizar la relación entre las personas del sistema, se creó primero la clase abstracta Persona, de la cual heredaron jerárquicamente Empleado y Pasajero mediante la palabra clave extends. Esto permitió establecer una estructura común para ambas clases (compartiendo datos básicos de identidad) y diferenciar sus características específicas mediante la herencia. Luego se desarrolló la clase Departamento, que incluye el nombre del departamento y un atributo lógico o enumerado que indica si este pertenece al sector aéreo o terrestre.

A continuación, se trabajó en la clase Aeropuerto, que representa una sede de la aerolínea y contiene información sobre su ubicación, como el país y la ciudad, además de una estructura de lista para almacenar a sus empleados. Se planteó una relación bidirección funcional entre Aeropuerto y Empleado: el aeropuerto mantiene una lista de las personas que trabajan ahí, mientras que cada empleado puede identificar el aeropuerto al que está asignado. Inicialmente se había considerado que los pilotos no tuvieran un aeropuerto de trabajo asignado, por lo que ese atributo podía ser nulo y no se incluiría en el constructor de Empleado. Sin embargo, esta decisión se descartó posteriormente y se estableció que todos los empleados tendrían un departamento de trabajo por igual. Por lo tanto, la idea inicial de diferenciar a los pilotos mediante la ausencia de una asignación de trabajo no se mantuvo como parte del diseño definitivo.

También se comenzó a definir la relación estructural entre las clases Avion y Asiento. Se estableció que un avión puede contener una colección de muchos asientos y que cada asiento pertenece a un único avión, formando una relación de uno a muchos (1-N) bidireccional en el código. Esto significa que el objeto Avion contiene una lista de sus asientos y que cada objeto

Asiento identifica al avión al que pertenece. Además, se diseñó un formato de identificación que permitiera reconocerlos con mayor facilidad, en lugar de utilizar un identificador numérico autoincremental convencional. El formato previsto combinaba el identificador del avión con el código del asiento, por ejemplo, AV-A320-02-1A. A pesar de definir esta estructura, se decidió que los identificadores no formarían parte de los constructores de Avion y Asiento, ya que la creación de estos registros no estará a cargo de este módulo del programa.

Finalmente, todas las clases desarrolladas hasta este momento cuentan únicamente con la definición de sus atributos privados, constructores base y sus respectivos métodos de acceso getters y setters. El trabajo de la jornada se concentró con éxito en establecer una arquitectura orientada a objetos clara, limpia y coherente con el esquema relacional de la base de datos, dejando preparada la base sobre la cual se implementarían las operaciones de conectividad, controladores y, posteriormente, las interfaces gráficas del sistema.

Fecha 21/09/2026

Durante esta jornada se consolidó la capa del modelo de negocio en Java mediante la implementación y refinamiento de las clases comerciales Servicio, Pago, CheckIn, Boleto y Vuelo. La arquitectura de software se diseñó bajo criterios de alta cohesión y bajo acoplamiento, priorizando que la consistencia de los datos financieros y temporales se resuelva de manera nativa en los constructores de las clases, reduciendo significativamente el riesgo de errores en tiempo de ejecución o discrepancias de caja.

La lógica aplicada al constructor de la clase Pago garantiza que la captura de los costos totales y de servicios sea síncrona al momento de la emisión del comprobante. Como se observa en la implementación del código, el constructor de Pago recibe las instancias de Pasajero y Boleto junto con el estado del pago, inicializando de forma inmediata los atributos costo_servicios y costo_total mediante llamadas directas a los métodos calcularPrecioServicios() y calcularPrecioTotal() encapsulados en el objeto Boleto. El método calcularPrecioServicios() realiza una iteración mediante un ciclo for-each sobre la colección dinámica l_servicios, acumulando secuencialmente el precio de cada servicio contratado obtenido mediante su respectivo método getter. Por su parte, calcularPrecioTotal() consolida el costo final sumando el precio_base del boleto al total acumulado de los servicios adicionales. Esta delegación de responsabilidades abstrae el cálculo matemático y resguarda la integridad histórica de la transacción frente a eventuales mutaciones posteriores en las tarifas del sistema.

```
public Pago(Pasajero pasajero, Boleto boleto, String estado) {
    this.pasajero = pasajero;
    this.boleto = boleto;
    this.estado = estado;
    this.costo_servicios = boleto.CalcularPrecioServicios();
    this.costo_total = boleto.CalcularPrecioTotal();
}
```

```
public double CalcularPrecioServicios() {
    double precio_total = 0;
    for (Servicio s : l_servicios) {
        precio_total += s.getPrecio();
    }
    return precio_total;
}

public double CalcularPrecioTotal() {
    return precio_base + CalcularPrecioServicios();
}
```

Asimismo, la vinculación automatizada de la fecha del boleto con la salida programada del avión (Vuelo) elimina la redundancia de datos y mitiga las fallas por ingreso manual o inconsistencias de calendario en el sistema. Finalmente, la inicialización preventiva de colecciones dinámicas mediante la incorporación de funciones operacionales como agregarServicio() o sus equivalentes agregarX() en todas las clases que contienen colecciones ArrayList asegura la estabilidad estructural del código y previene excepciones de tipo NullPointerException previo a su acople con la interfaz gráfica definitiva.

En cuanto a la interfaz de depuración, se integraron interfaces de prueba rápidas utilizando la librería javax.swing.JOptionPane mediante funciones de diagnóstico mostrarX() en aquellas clases que son instanciadas o llamadas desde otras (por ejemplo, Asiento, que contiene una referencia explícita al objeto Avion). Por el contrario, la clase Pago se omitió de esta lógica temporal al no ser requerida de forma directa por otras dependencias en esta fase del modelo. Se decidió deliberadamente que estos métodos de prueba no rendericen de forma recursiva las clases relacionadas que contienen; por ejemplo, aunque Avion posee una lista interna de objetos Asiento, estos no se despliegan en el panel emergente de diálogo. Esta restricción responde a una planificación arquitectónica a futuro, donde la visualización y manipulación de colecciones complejas se delegará formalmente a componentes JFrame y JTable para una presentación de datos más adecuada y limpia para el usuario.

Por otra parte, se modificó la estructura de los asientos de los aviones dentro del modelo de negocio. Inicialmente, cada avión contaba con una capacidad de 186 asientos, pero se decidió reducirla a 180 eliminando seis unidades. El motivo técnico principal fue facilitar la posterior organización visual y lógica del diagrama de selección de asientos en Java. Con esta nueva distribución, el inventario se estructuró en dos bloques simétricos de 90 asientos, dispuestos

en una matriz homogénea de 6 columnas por 15 filas para cada grupo, optimizando su futura implementación en los componentes de la interfaz gráfica. En sintonía con esto, se amplió la lista de servicios disponibles en el sistema incorporando la "Selección de Asiento" con un valor comercial de 15.000, permitiendo la reserva manual por parte del pasajero en lugar de una asignación aleatoria, operando bajo la misma lógica centralizada de los servicios de la aerolínea.

Finalmente, se realizaron modificaciones estructurales en el código SQL de la base de datos para acompañar de forma paralela la evolución del programa en Java. En la tabla de pagos se incorporaron los campos boleto_id, costo_servicios, costo_total y estado, junto con sus respectivas restricciones de clave foránea (FOREIGN KEY) para garantizar la integridad referencial con las entidades Pasajero y Boleto. El atributo estado se restringió rígidamente mediante un tipo de dato ENUM que contempla las opciones: Pagado, Pendiente, Cancelado, Robado, Fiado, Reducido, Detenido y Devuelto. Asimismo, se reestructuró la tabla CheckIn para incluir las referencias cruzadas hacia las entidades Pasajero, Boleto, Vuelo y Pago, sumando además el campo puerta_embarque para su correcta trazabilidad en el aeropuerto. La jornada concluyó con la actualización integral de todos los scripts INSERT del sistema, reflejando fielmente la nueva capacidad de las aeronaves, el nuevo servicio comercial disponible y las modificaciones relacionales introducidas.

Fecha 22/09/2026

Vinculación del Driver de Persistencia e Implementación de JDBC:

Como hito fundamental para iniciar la comunicación con el motor de base de datos, se incorporó al classpath del proyecto la librería externa MySQL JDBC Driver

(mysql-connector-java-5.1.23-bin.jar). Esta dependencia actúa como el conector oficial para permitir que la aplicación Java traduzca e intercambie instrucciones nativas con el servidor local MySQL. A partir de esta adición, se dio inicio al desarrollo del patrón DAO (Data Access Object) y la gestión de conexiones relacionales creando la clase utilitaria ConexionSQL. Esta clase utilitaria centraliza los parámetros esenciales de acceso, tales como la URL de conexión, el usuario (USER) y la contraseña (PASSWORD), hacia la base de datos local denominada Chorifly a través del método DriverManager.getConnection(). Toda esta lógica se encapsuló minuciosamente mediante bloques de control try-catch para capturar excepciones de tipo SQLException y garantizar el retorno seguro de un objeto activo de tipo Connection.

Para corroborar el correcto enlace con el servidor utilizando el conector agregado antes de proceder con las operaciones de escritura, se modificó la clase principal del programa, Chorifly. Dentro de su método ejecutable main, se instanció el controlador de la aerolínea y se programó una subrutina de prueba para evaluar el estado del objeto de conexión mediante una estructura condicional (if-else). Esta validación emite mensajes informativos de control por consola ("Conexión Exitosa." o "Error de Conexión.") según corresponda, verificando la viabilidad del canal de comunicación y la correcta respuesta del socket del servidor antes de iniciar la persistencia formal.

En consonancia con este flujo de datos, se programó la clase UsuarioDAO, encargada de aislar la lógica de persistencia de la entidad Usuario respecto de la lógica del negocio. Se implementó el método específico insertar(Usuario u), el cual emplea sentencias parametrizadas mediante la interfaz PreparedStatement para realizar inserciones de datos de manera segura y eficiente, previniendo de forma nativa vulnerabilidades críticas de inyección

SQL. Este método mapea los atributos privados del objeto de Java (incluyendo el campo email añadido recientemente y la bandera de privilegios de administración) hacia las columnas correspondientes de la tabla de la base de datos mediante métodos setString() y setBoolean(), ejecutando finalmente la transacción mediante la llamada sentencia_sql.executeUpdate() y controlando de manera estructurada cualquier anomalía mediante la captura de excepciones SQLException.

Inyectado de forma paralela al despliegue de la conectividad en Java, se realizaron refactorizaciones críticas en la estructura y registros de la base de datos SQL para subsanar inconsistencias nominales y flexibilizar el almacenamiento. En primer lugar, se corrigió un error de escritura dentro de los registros de departamentos comerciales mediante un script de actualización, reemplazando la denominación heredada en inglés “Pilot” por su equivalente en español “Piloto”, normalizando así el idioma de los datos maestros del sistema.

A nivel de esquema estructural, se alteró la tabla Usuario mediante una sentencia ALTER TABLE para ampliar la longitud máxima del campo contraseña_usuario de VARCHAR(15) a VARCHAR(30), garantizando una mayor tolerancia de caracteres para las políticas de seguridad de las claves de acceso. Asimismo, en la tabla Pasajero se reestructuró la definición de la clave primaria, renombrándola de id_pasajero a IDPasajero en pos de mantener criterios uniformes de nomenclatura CamelCase con el resto de las entidades de la base de datos. En sintonía con esta estandarización, se modificó la restricción de integridad de la clave foránea usuario_id para corregir su referencia, apuntando explícitamente hacia la columna Usuario(IDUsuario) en lugar de la definición previa usuario(id_usuario). Estas oportunas intervenciones eliminaron de raíz las discrepancias relacionales entre el diccionario de datos

de SQL y el mapeo de objetos en Java, consolidando una arquitectura sólida para el desarrollo de las siguientes fases de desarrollo.

Fecha 24/09/2026

Durante esta jornada se expandió significativamente la arquitectura de persistencia de la aplicación mediante el desarrollo de las clases especializadas VueloDAO, EmpleadoDAO, PasajeroDAO y BoletoDAO. El diseño conceptual y la estructura inicial de estos componentes se implementaron utilizando el material de referencia provisto en el Classroom, aunque se tomó la determinación de que en jornadas posteriores se migraría hacia un formato de funciones auxiliares y métodos genéricos que optimicen el mantenimiento del código. El objetivo central de la sesión fue implementar los métodos insertar() para cada entidad, finalizando por completo esta función en EmpleadoDAO y VueloDAO.

A nivel arquitectónico, estos Objetos de Acceso a Datos (DAO) actúan como una capa intermedia de abstracción encargada de centralizar y aislar la lógica de comunicación con el motor de la base de datos relacional. Su funcionamiento general radica en recibir los objetos de negocio de la aplicación, gestionar los recursos de conexión y traducir los datos del entorno Java hacia sentencias SQL legibles por el motor. En esta etapa del diseño, las operaciones se estructuraron de manera directa dentro de cada función principal; cada método asume de forma compacta tanto la apertura del canal físico mediante el bridge de conexión como la preparación, parametrización y ejecución de la consulta correspondiente.

A nivel de diseño de software, un desafío clave resuelto fue el desacoplamiento y mapeo de atributos basados en objetos complejos hacia campos primitivos o claves foráneas. En la clase VueloDAO, se tradujeron las referencias de los objetos internos de tipo Avion y Aeropuerto hacia sus correspondientes tipos relacionales (String e int) mediante métodos de acceso anidados del tipo getAvion().getIDAvion(). Durante este proceso, se detectó que el driver JDBC de MySQL provisto no posee soporte directo para el método específico setLocalDateTime() en la interfaz PreparedStatement. Para solucionar esta limitación, se optó por utilizar el método setObject() de forma genérica pasándole los atributos temporales correspondientes, garantizando una compatibilidad transparente con las marcas temporales de Java.

En EmpleadoDAO y PasajeroDAO se resolvió la persistencia de relaciones de herencia y agregación hacia las tablas Departamento, Aeropuerto y Usuario, adaptando sincrónicamente los formatos de fechas de la entidad Pasajero para su inserción segura. Asimismo, en BoletoDAO se inició el desarrollo de la estructura base aislando el identificador del Pasajero y preparando los parámetros necesarios para persistir las transacciones comerciales de tickets. Para estandarizar el flujo de control, excepciones y confirmaciones en la interacción con el usuario, se integró la librería javax.swing.JOptionPane. Cada bloque try-catch encargado de capturar fallas de tipo SQLException fue estructurado para utilizar el método estático showMessageDialog(), desplegando ventanas emergentes de tipo INFORMATION_MESSAGE o ERROR_MESSAGE tanto para confirmar el éxito de la transacción ("Guardado Exitosamente") como para reportar detalles técnicos controlados ante fallas operacionales en la base de datos.

Por fuera del entorno de desarrollo de software, se solucionó un problema crítico de conectividad en el esquema de redes del proyecto. Se detectó un conflicto que provocaba que, al emplear un enrutamiento estático estricto, las solicitudes dinámicas de asignación de direcciones IP mediante DHCP no fueran procesadas a través de los diferentes segmentos de red. Para permitir la correcta propagación de los paquetes de difusión (broadcast), se intervinieron los enrutadores de la topología añadiendo los comandos "ip route" para consolidar los caminos estáticos y la directiva "ip helper-address" en las interfaces de puerta de enlace (Gateway) correspondientes, logrando redirigir de forma dirigida las peticiones DHCP hacia el servidor central del sistema.

Imagen de como se solucionó el error antes mencionado.

```
sentencia_sql.setString(1, v.getAvionUsado().getId_avion());
sentencia_sql.setInt(2, v.getAeropuertoOrigen().getId_aeropuerto());
sentencia_sql.setInt(3, v.getAeropuertoDestino().getId_aeropuerto());
sentencia_sql.setLocalDateTime(4, v.getFechaHoraSalida());
sentencia_sql.setLocalDateTime(5, v.getFechaHoraLlegada());
sentencia_sql.setString(6, v.getEstado());
```

```
sentencia_sql.setString(1, v.getAvionUsado().getId_avion());
sentencia_sql.setInt(2, v.getAeropuertoOrigen().getId_aeropuerto());
sentencia_sql.setInt(3, v.getAeropuertoDestino().getId_aeropuerto());
sentencia_sql.setObject(4, v.getFechaHoraSalida());
sentencia_sql.setObject(5, v.getFechaHoraLlegada());
sentencia_sql.setString(6, v.getEstado());
```

Fecha 25/09/2026

Durante esta jornada se completó la arquitectura de persistencia para el conjunto principal de entidades del sistema, estructurando las operaciones fundamentales de manipulación de datos en la base de datos relacional de Chorifly. El trabajo se focalizó en codificar e integrar de manera definitiva los métodos insertar() para las clases CheckInDAO, PagoDAO y EmpleadoDAO, consolidando su capacidad de almacenamiento seguro en el motor SQL.

Asimismo, se logró implementar un flujo dual de persistencia mediante los métodos insertar() y listar() para las entidades críticas BoletoDAO, PasajeroDAO, UsuarioDAO y VueloDAO, logrando que estas últimas no solo registren información, sino que también recuperen de forma dinámica los registros existentes desde el servidor. Con este avance, se concluyó formalmente la primera fase de desarrollo de los objetos de acceso a datos (DAO) que requieren operaciones de inserción. Esto marcó el inicio de una metodología de trabajo estructurada por secciones operacionales en lugar de un enfoque aislado clase por clase. El flujo metodológico estandarizado para las siguientes etapas del proyecto consistirá en abordar secuencialmente los bloques de inserción, selección general (Select *), selección específica parametrizada (Buscar), actualización (Update) y borrado lógico o físico (Delete).

A nivel de diseño de software y persistencia, el desarrollo de la lógica de inserción requirió la parametrización estricta de las sentencias SQL a través del uso de PreparedStatement. Se mapearon minuciosamente los atributos de los objetos de negocio hacia los tipos de datos nativos de la base de datos, asegurando la correspondencia exacta de cadenas de texto, valores booleanos y tipos numéricos.

El principal desafío de la sesión radicó en el diseño de la función listar(), orientada a la extracción y lectura de datos en memoria dinámica. Para cumplir con este requerimiento, se estructuró la ejecución de consultas de selección general ("SELECT *") gestionadas mediante interfaces ResultSet. La lógica de control fue programada para iterar de forma secuencial sobre el puntero de resultados mediante el método .next() dentro de bucles condicionales, extrayendo los identificadores, nombres, credenciales y estados lógicos de cada registro para su posterior procesamiento y empaquetado en colecciones de objetos.

Inmerso en la interfaz y el flujo de confirmaciones, las nuevas funciones desarrolladas se acoplaron al componente javax.swing.JOptionPane para volcar los datos recuperados desde la base de datos de manera directa y ordenada en ventanas emergentes de tipo INFORMATION_MESSAGE. De igual forma, se dio continuidad al tratamiento de fallas centralizado, garantizando que ante cualquier anomalía en el driver JDBC o en la sintaxis SQL, el sistema intercepte la excepción en el bloque catch correspondientemente y despliegue ventanas de alerta visualizables.

En paralelo al entorno de desarrollo de la aplicación, se ejecutó una exhaustiva auditoría y refactorización sobre el código SQL del motor de la base de datos para garantizar su estabilidad y mitigar fallos en tiempo de ejecución. Se subsanaron errores sintácticos y de escritura en los scripts DDL, tales como comas huérfanas y delimitadores incorrectos que interrumpían la creación secuencial de las tablas.

Asimismo, se normalizó el esquema de datos actualizando la capacidad de almacenamiento en la entidad Avion. Este valor se ajustó de 186 a 180 asientos para reflejar con precisión matemática la reducción realizada en jornadas previas, orientada a la optimización del

diagrama visual de selección de asientos en la interfaz de usuario. Para robustecer la integridad referencial y de dominio, se incorporaron restricciones de unicidad (UNIQUE) en atributos críticos como el correo electrónico y el nombre de usuario de la entidad Usuario, impidiendo la duplicidad de registros lógicos en el sistema.

Finalmente, se aplicaron buenas prácticas de estandarización en los scripts de la base de datos, unificando el uso de mayúsculas sostenidas en las palabras clave y tipos de datos SQL. Se mutaron declaraciones primitivas como varchar e int hacia sus equivalentes formales VARCHAR e INT, consolidando un repositorio de persistencia limpio, legible y completamente uniforme.

Fecha 27/09/2026

Durante esta jornada se actualizaron significativamente el Diagrama Entidad-Relación (DER) del proyecto Chorifly en su entidad Asiento, procediendo al reemplazo del atributo numérico "numero" por el campo alfanumérico "codigo". Esta modificación estructural responde a una necesidad del modelado de negocio, dado que las ubicaciones físicas dentro de las aeronaves combinan caracteres numéricos y alfabéticos (por ejemplo, "01A"), lo cual imposibilitaba su almacenamiento bajo un tipo de dato puramente numérico (como INT) y requería garantizar la integridad de los datos desde la capa de persistencia en SQL mediante un tipo VARCHAR.
A nivel de arquitectura de software, se llevó a cabo una refactorización crítica en la capa de Objetos de Acceso a Datos (DAO) con el objetivo de modularizar el código fuente y erradicar la redundancia estructural identificada en sesiones previas. Para lograr este desacoplamiento, la lógica operativa interna de persistencia se descentralizó mediante la implementación de dos funciones auxiliares especializadas en el manejo de flujos de entrada y salida.

Por un lado, se diseñó la función insert para recibir de forma genérica el objeto de negocio y una sentencia preparada (PreparedStatement), abstrayendo el seteo ordenado y parametrizado de cada atributo antes de consolidar el impacto en el motor relacional mediante la ejecución de la consulta. Por otro lado, se implementó la función select orientada a procesar el puntero del cursor de la consulta (ResultSet), extrayendo dinámicamente los datos de las columnas correspondientes para mapearlos y transformarlos en una estructura de cadena de texto formateada o en la instanciación de objetos de entidad, disponibles para su posterior consumo en la interfaz gráfica de usuario.

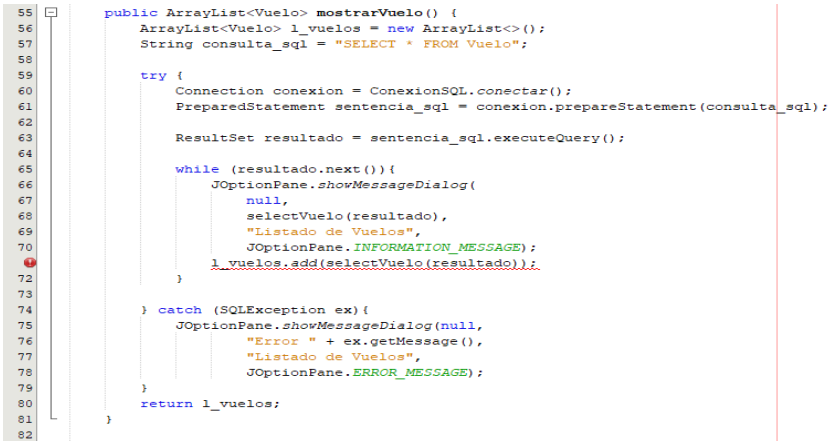
A partir de esta abstracción base, las clases operacionales definitivas consolidaron su estructura interna según sus respectivas restricciones lógicas. Los componentes BoletoDAO, CheckInDAO, EmpleadoDAO, PagoDAO, PasajeroDAO, UsuarioDAO y VueloDAO completaron la implementación de su ciclo CRUD mediante el desarrollo formal de los métodos insert, select, agregar, mostrar, buscar, editar y eliminar. En contraste, AsientoDAO incorporó exclusivamente las operaciones select, mostrar, buscar y editar. La omisión deliberada de las funciones de inserción y eliminación independiente obedece a una restricción del dominio del negocio, donde las plazas dependen estrictamente de la preexistencia y configuración física de un avión, careciendo de un ciclo de vida autónomo en el sistema.

Debido a esta dependencia lógica y jerárquica, AsientoDAO constituyó el último componente específico por tabla en ser codificado. Esta disposición arquitectónica permitió delimitar con éxito el alcance de la capa actual y sentar las bases de diseño para la futura implementación de ExtrasDAO, una clase proyectada específicamente para centralizar la gestión de aquellas

tablas accesorias que solo requieran operaciones puntuales y acotadas sin sobrecargar el árbol de directorios de fuentes.

Finalmente, durante las pruebas de integración y lectura ejecutadas sobre VueloDAO, se detectó una falla técnica crítica dentro del método mostrarVuelo(). Específicamente, en la instrucción l_vuelos.add(selectVuelo(resultado)), la función auxiliar selectVuelo retorna un tipo de objeto o formato incompatible con la firma y estructura de tipado del ArrayList que se pretende poblar en memoria. Ante este escenario de error de coincidencia de tipos (Type Mismatch) en tiempo de compilación, se procedió al registro y adjunción de la captura del segmento de código afectado en la documentación técnica, planificando su resolución prioritaria para las próximas jornadas de desarrollo.

Se adjunta imagen del error antes mencionado:



```
55      public ArrayList<Vuelo> mostrarVuelo() {
56          ArrayList<Vuelo> l_vuelos = new ArrayList<>();
57          String consulta_sql = "SELECT * FROM Vuelo";
58
59          try {
60              Connection conexion = ConexionSQL.conectar();
61              PreparedStatement sentencia_sql = conexion.prepareStatement(consulta_sql);
62
63              ResultSet resultado = sentencia_sql.executeQuery();
64
65              while (resultado.next()){
66                  JOptionPane.showMessageDialog(
67                      null,
68                      selectVuelo(resultado),
69                      "Listado de Vuelos",
70                      JOptionPane.INFORMATION_MESSAGE);
72                  l_vuelos.add(selectVuelo(resultado));
73              }
74          } catch (SQLException ex){
75              JOptionPane.showMessageDialog(null,
76                      "Error " + ex.getMessage(),
77                      "Listado de Vuelos",
78                      JOptionPane.ERROR_MESSAGE);
79          }
80          return l_vuelos;
81      }
82
```

Fecha 28/09/2026

Durante esta jornada se corrigió formalmente el error de compatibilidad técnica y tipado detectado previamente en la clase VueloDAO. Se modificó el método mostrarVuelo() eliminando el problema intermedio que producía la diferencia de tipos de datos en tiempo de compilación. A partir de esta intervención, la función mapea e incorpora de forma directa los registros devueltos por la función auxiliar selectVuelo(resultado) al objeto de tipo ArrayList<Vuelo>, garantizando el retorno correcto de la colección hacia los componentes que la requieran.

A nivel de diseño de software, se creó e implementó formalmente la clase especializada ExtrasDAO en el paquete de persistencia. Esta clase fue diseñada con el propósito de centralizar en un único componente aquellas consultas de selección y búsquedas específicas por identificador único, implementando métodos específicos tales como buscarAvionPorId, buscarAeropuertoPorId y buscarDepartamentoPorId. El enfoque estuvo dirigido a optimizar la estructura del proyecto simplificando la lógica de las tablas secundarias que poseen pocas funciones y que en el modelo del negocio no necesitan un ciclo CRUD completo con operaciones autónomas de creación, edición o borrado. Esta estrategia de organización evitó la creación innecesaria de muchos archivos DAO pequeños dentro del código fuente, mejorando la modularidad. Cabe destacar que esta solución de diseño fue presentada al docente, quien aprobó formalmente este criterio y la centralización de las funciones.

Técnicamente, la clase ExtrasDAO resuelve con éxito la reconstrucción y el mapeo inverso de objetos complejos junto con sus claves foráneas asociadas desde el modelo relacional hacia el orientado a objetos. Al momento de recuperar entidades del sistema que tienen

dependencias jerárquicas como es el caso de Pasajero, Boleto o Pago, el DAO llama internamente a sus propios métodos de búsqueda para cargar de manera directa las referencias de los objetos relacionados. Este procedimiento permite, por ejemplo, incluir directamente el objeto de tipo Usuario dentro de la instancia de Pasajero, o vincular las instancias activas de Pasajero y Vuelo al momento de crear un Boleto en memoria, logrando mantener la integridad total de los datos desde las consultas de la aplicación.

Fecha 29/09/2026

Durante esta jornada se inició la etapa de desarrollo de la interfaz gráfica de usuario (GUI) mediante el diseño inicial de las pantallas V_InicioSesion, V_MenuPrincipal y V_MenuPrincipalAdmin. En esta fase preliminar, las ventanas se enfocaron estrictamente en la maquetación visual y la disposición de componentes dentro de los formularios.

A nivel de lógica conceptual, se diseñó la estructura del control de accesos para el inicio de sesión. La regla establece que el sistema verificará en primera instancia si las credenciales corresponden al Administrador para redirigirlo a su menú específico (V_MenuPrincipalAdmin). En caso negativo, evaluará si posee privilegios de empleado normal para conceder el acceso a V_MenuPrincipal; si no cumple con ninguna de las dos condiciones, se denegará la entrada. Cabe destacar que el flujo de navegación y la destrucción o visibilidad de las ventanas para evitar conflictos de cambio de menú en caliente aún no han sido programados, limitándose la sesión únicamente al diseño visual de las maquetas.

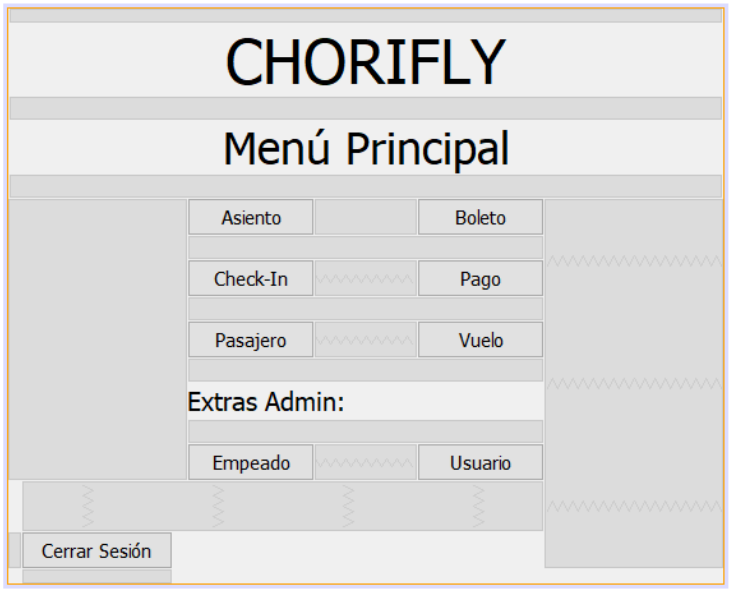
A nivel de arquitectura de software, se realizó una reforma integral sobre los constructores de las clases de negocio del modelo para mejorar el flujo de datos proveniente de la capa de persistencia. Para todas las entidades que se recuperan desde la base de datos, tales como Asiento, Avion, Aeropuerto, entre otras, se incorporaron constructores vacíos por cuestiones de comodidad operativa dentro de las funciones de los DAO. Esta decisión permite crear objetos limpios de forma secuencial y cargar sus atributos dinámicamente mediante métodos setters a medida que se procesa el puntero de resultados del ResultSet.

En contraposición, las clases vinculadas a objetos DAO que requieren transacciones de inserción activa en la base de datos, con la excepción de AsientoDAO, cuya naturaleza es inmutable e independiente, conservaron o incorporaron constructores parametrizados completos. Dado que estas entidades sí están diseñadas para ser creadas desde la aplicación por el usuario, disponer de un constructor con parámetros facilita el empaquetamiento directo de los datos obligatorios en una sola línea de código al momento de la instanciación, agilizando el desarrollo en la capa visual y de control.

Imágenes de los Menús Principales







Fecha 30/09/2026

Durante esta jornada se desarrollaron y pusieron en funcionamiento las interfaces gráficas V_InicioSesion y V_CrearCuenta, aplicando una línea visual unificada basada en una paleta de colores con tonos azul, celeste y turquesa a través de paneles contenedores JPanel colocados de forma jerárquica sobre el marco principal JFrame. Cabe destacar que al terminar

esta sesión la actualización del comportamiento y la lógica interna de los menús principales quedó anotada como pendiente para su resolución en las etapas inmediatas del proyecto.

A nivel de comportamiento del programa, la lógica de funcionamiento del método btnIngresarActionPerformed() en la ventana V_InicioSesion fue estructurada y validada bajo reglas de control claras. En la evaluación de credenciales el flujo inicia consultando el registro del administrador y la lista de empleados a través de la capa UsuarioDAO. El sistema revisa primero si los datos ingresados coinciden con el perfil de administrador, procediendo a abrir la interfaz V_MenuPrincipalAdmin y a cerrar la ventana actual mediante la instrucción this.dispose(). Si los datos no son del administrador, se realiza un recorrido sobre la lista de empleados para verificar la validez del acceso y abrir la pantalla V_MenuPrincipal. Para el control de bloqueo, ante la detección de un ingreso incorrecto, se baja en uno un contador de intentos local. Si este contador llega a cero, el sistema muestra una alerta de error mediante un cuadro de diálogo emergente de JOptionPane y cierra de forma inmediata el programa a través de la instrucción System.exit(0).

Por otra parte, se programó formalmente el método registrarUsuario() dentro de la ventana de creación de cuentas, el cual centraliza las revisiones de seguridad previas a guardar los datos en la base de datos. En los filtros de seguridad se incluye la limpieza de los campos de texto mediante el método trim() para borrar espacios en blanco innecesarios y se realiza una comprobación de coincidencia exacta entre la contraseña escrita y su confirmación. Para las restricciones de unicidad, el sistema llama al mismo tiempo a las funciones existeUsuario() y existeEmail() de la capa DAO para verificar si esos valores ya existen en la base de datos, deteniendo el proceso mediante ventanas de error si se encuentra que los datos están duplicados. Una vez superados con éxito todos los filtros, se crea el objeto Usuario

guardando por defecto los privilegios de empleado en verdadero para asegurar su registro definitivo a través del método insertar().

Diseño de la ventana principal (Inicio de Sesión)



Fecha 02/10/2026

Durante esta jornada se implementó la arquitectura de control y seguridad para el proceso de alta de cuentas mediante la creación de la clase especializada Validaciones. El objetivo principal de este componente fue separar las reglas de negocio de la capa de interfaz gráfica de usuario, asegurando la integridad y el formato de los datos antes de interactuar con los métodos de la capa de persistencia.

A nivel de desarrollo de software, el método validarCuenta() incorporó filtros de validación y mecanismos de control de formato basados en reglas específicas. Para los identificadores de usuario, se limitó el ingreso del campo mediante técnicas de coincidencia de patrones (Pattern Matching), admitiendo de manera exclusiva caracteres alfabéticos, tildes y guiones bajos, bloqueando el uso de valores numéricos o espacios en blanco.

Para los formatos de correo, se configuró una expresión regular (Regex) insensible a mayúsculas y minúsculas para forzar el cumplimiento de una estructura de correo electrónico válida bajo el dominio de gmail.com. Con respecto a los campos obligatorios, se programaron condicionales lógicos acoplados a consultas de verificación en tiempo real como existeUsuario() y existeEmail() vinculadas directamente a la clase UsuarioDAO, con el fin de validar la disponibilidad y unicidad de las credenciales en la base de datos del proyecto Chorifly.

En paralelo, se ejecutaron modificaciones sobre el esquema relacional del sistema. Se alteró la estructura de la tabla Usuario en el motor de la base de datos de Chorifly agregando las columnas pregunta_seguridad y respuesta_seguridad. El propósito central de esta modificación fue preparar el almacenamiento necesario para la posterior implementación de la función de recuperación de credenciales en Java, permitiendo a los usuarios del sistema restablecer sus accesos mediante este mecanismo de autenticación secundaria.

A raíz de esta reestructuración en la base de datos, se expandió el modelo de la clase entidad Usuario y su correspondiente lógica de mapeo en la capa DAO. Los nuevos atributos de seguridad fueron agregados directamente al constructor con parámetros principal del objeto de negocio, logrando una sincronización exacta entre las variables del entorno orientado a objetos y las nuevas especificaciones de la base de datos.

Por otra parte, se avanzó en la maquetación y diseño de la interfaz VentanaRecuperarCuenta. Durante esta sesión, la ventana quedó lista con la distribución visual y los componentes Swing requeridos, quedando limitada en su funcionamiento únicamente a la acción de regreso

mediante el botón volver, dejando el desarrollo de la lógica interna para el botón de recuperación de cuenta para las próximas jornadas.

Finalmente, se integró la librería externa de interfaz de usuario flatlaf-3.2.jar en la ventana principal del proyecto Chorifly. La inicialización de este entorno dentro de su método principal main a través de la instrucción FlatLightLaf.setup() permitió heredar propiedades avanzadas de diseño sobre los campos de texto de los formularios, logrando incorporar de forma directa la propiedad de texto de ayuda (conocido técnicamente como Placeholder o Prompt Text), el cual se oculta automáticamente al iniciar la escritura con el teclado.

Imagenes de las Ventanas de Crear Cuenta y Recuperar Cuenta





Fecha 05/10/2026

Durante esta jornada se completó la lógica de funcionamiento para el módulo de restablecimiento de accesos mediante la programación de los métodos recuperarCuenta() y actualizarContraseña(), consolidando el comportamiento y el flujo de navegación entre las interfaces VentanaRecuperarCuenta y VentanaRecuperarContraseña. El sistema se diseñó bajo un esquema de control estricto, limitando el flujo de autenticación exclusivamente a la validación del correo electrónico y la respuesta de seguridad preestablecida, sin permitir vías alternativas en el proceso.

A nivel de desarrollo de software, se realizó una reforma importante sobre los scripts, clases y componentes existentes con el objetivo de estandarizar los nombres de todo el código fuente del proyecto. Se modificó la escritura de las variables del sistema, reemplazando el estilo de snake_case por el estándar CamelCase para asegurar la prolijidad y la alineación con las convenciones de diseño y buenas prácticas del entorno Java. A partir de este hito en la arquitectura, todas las declaraciones siguientes adoptarán esta regla de escritura de forma obligatoria.

A nivel de comportamiento del programa, el método recuperarCuenta() quedó estructurado bajo reglas claras de control y validación de identidad. En el filtrado de registros, el sistema busca la lista de usuarios a través de la capa UsuarioDAO y realiza una comprobación para asegurar la existencia del correo electrónico ingresado en el campo de texto txtEmailRecuperar. Ante un correo válido, se abre un cuadro de diálogo interactivo mediante el método JOptionPane.showInputDialog(), el cual muestra de forma dinámica una ventana emergente con la pregunta de seguridad recuperada desde la base de datos.

Para la gestión de intentos, se configuró un margen de tres intentos para responder la pregunta. Si el usuario cancela la entrada de datos retornando un valor nulo, el proceso se interrumpe de forma segura. En caso de coincidencia exitosa con el registro base aplicando el método equalsIgnoreCase() para ignorar mayúsculas y minúsculas, se abre la ventana siguiente pasando el objeto de negocio validado como parámetro y se cierran los recursos de la interfaz actual. Si se agota el cupo de intentos fallidos, se bloquea el acceso y se fuerza el cierre total del programa mediante la instrucción System.exit(0).

Por otra parte, se programó el método actualizarContraseña() dentro de la clase VentanaRecuperarContraseña para hacer efectiva la modificación en la base de datos. Primero se revisan las reglas de negocio invocando los métodos de la clase Validaciones para asegurar que las nuevas credenciales cumplan con los requisitos y mantengan una igualdad exacta entre sí. Al superar las pruebas de validación, se crea un objeto de tipo Usuario en el cual se introduce el identificador original recuperado junto con la nueva clave, ejecutando la actualización en la base de datos a través del método especializado editarContraseña() de la capa DAO. Tras confirmar el éxito del cambio, el sistema cierra de forma segura la pantalla actual y redirige al usuario de regreso hacia la ventana de inicio de sesión VentanaInicioSesion.

A nivel de lógica de control, se expandió y reforzó la clase Validaciones mediante la inclusión de restricciones adicionales y el desarrollo del nuevo método validarContraseña(), logrando centralizar las reglas de seguridad. Dentro del método validarCuenta(), se incorporó una validación mediante expresiones regulares sobre las claves de usuario para limitar la entrada a letras, números y un conjunto de caracteres especiales específicos, previniendo la inyección de símbolos no soportados durante el registro de cuentas. El nuevo método

validarContraseña() fue diseñado específicamente para dar soporte seguro al proceso de restablecimiento de credenciales de la ventana VentanaRecuperarContraseña. Esta función revisa en primer lugar que el campo no se encuentre vacío y valida mediante el patrón de expresión regular que el texto solo contenga caracteres permitidos de tipo alfanumérico o símbolos especiales aprobados. Además, se programó una restricción de longitud mínima obligatoria mediante la evaluación del tamaño de la cadena con contraseña.length() >= 8, rechazando el cambio con un mensaje de error si el usuario ingresa una clave más corta. Finalmente, el método realiza la comparación lógica de igualdad entre los textos de la contraseña y su repetición para comprobar su paridad, devolviendo un valor verdadero únicamente cuando la estructura supera con éxito todos los filtros del negocio.



Fecha 07/10/2026

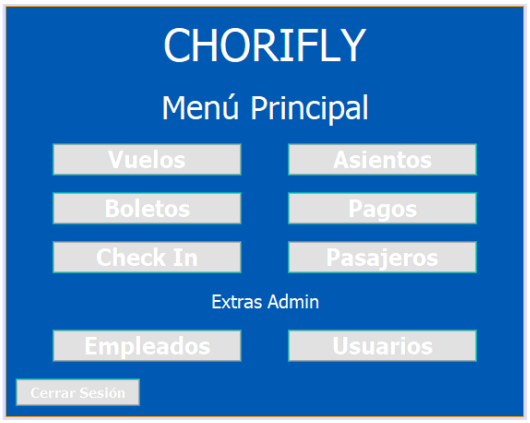
Durante esta jornada se inició formalmente la etapa de administración del sistema mediante el desarrollo y la puesta en funcionamiento de las interfaces gráficas VentanaCrudVuelos y VentanaCrudAsientos. El objetivo principal de estos componentes fue maquetar los paneles visuales y estructurar los elementos Swing que permitirán al usuario con privilegios de administrador gestionar los itinerarios de vuelo y controlar la ocupación de las plazas de las aeronaves de Chorifly de manera directa desde la aplicación.

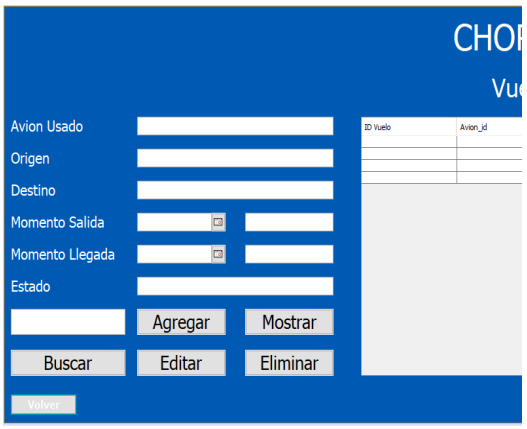
A nivel de comportamiento del programa, la ventana VentanaCrudVuelos fue diseñada para centralizar la visualización y edición de las rutas aéreas. La interfaz incluye una tabla principal montada sobre un componente JTable para desplegar de forma ordenada los registros de vuelos activos recuperados desde la base de datos a través de la capa VueloDAO. Asimismo, se integraron campos de texto específicos y selectores de fecha para capturar datos esenciales como el aeropuerto de origen, el aeropuerto de destino, el avión asignado y las marcas temporales de salida y llegada, dejando preparada la lógica para invocar las funciones de actualización y borrado lógico en las próximas sesiones de desarrollo.

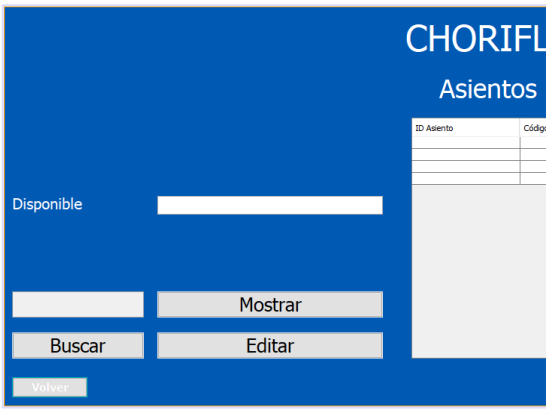
Por otra parte, se avanzó en la codificación de la pantalla VentanaCrudAsientos para dar soporte al control de ubicaciones dentro de los aviones comerciales. Siguiendo el criterio de diseño establecido previamente para el inventario simétrico de las aeronaves, se estructuró un panel visual orientado a facilitar la selección y la actualización del estado de los asientos.

Esta pantalla interactúa con la capa de persistencia mediante los métodos de búsqueda por identificador de la clase AsientoDAO, lo cual permitirá cargar dinámicamente la configuración del mapa de asientos correspondiente al modelo de avión seleccionado en la interfaz.

Finalmente, el flujo de navegación general de la capa gráfica fue enlazado con la interfaz del menú de administración VentanaMenuPrincipalAdmin. Se programaron los eventos de acción para los botones de control de este menú, logrando que al interactuar con las opciones de gestión de vuelos o de asientos se instancien correctamente las ventanas CRUD desarrolladas durante la sesión, al mismo tiempo que se liberan los recursos de la pantalla anterior mediante la instrucción dispose() para mantener una transición limpia y ordenada entre los diferentes formularios del sistema.







Fecha 08/10/2026

Durante esta jornada se mejoró la capa de seguridad y el control de la lógica de negocio mediante la integración de un sistema de validación de fechas y de dominios de correo, evitando fallos lógicos antes de guardar los cambios en la base de datos del proyecto Chorifly. Asimismo, se modificó la estructura interna del componente ExtrasDAO para optimizar el manejo de las conexiones en el motor relacional.

A nivel de comportamiento del programa y desarrollo en el entorno Java, las actualizaciones aplicadas se estructuraron bajo reglas técnicas bien definidas. Para el control cronológico, se creó e incorporó el método validarFecha() dentro de la clase Validaciones. Este proceso realiza una comparación de tiempos utilizando el método nativo isAfter(), devolviendo un valor falso si la fecha y hora de salida es posterior a la de llegada.

Esta regla de control de fechas se colocó de forma directa dentro de los métodos btnAgregarActionPerformed() y btnEditarActionPerformed() de la clase VentanaCrudVuelos. Al detectar un error de tipo DateTimeException o recibir una respuesta negativa del validador, el proceso se detiene y muestra un mensaje de alerta mediante JOptionPane.ERROR_MESSAGE, impidiendo que se guarden datos incorrectos en la base de datos.

Para la validación de dominios, dentro del método validarCuenta() se amplió la revisión de los correos de usuario mediante una estructura condicional doble. El patrón de expresiones regulares se modificó para aceptar correos corporativos que coincidan con el dominio de

chorifly.com, manteniendo al mismo tiempo la compatibilidad con el dominio comercial de gmail.com y rechazando mediante avisos cualquier formato diferente.

A nivel de persistencia en la capa DAO, se unificó el uso de bloques try-with-resources para la declaración y apertura de las interfaces Connection y PreparedStatement en todos los métodos de búsqueda, tales como buscarAsientoPorId() y buscarAvionPorId().

Este cambio de diseño asegura que los recursos abiertos para realizar las consultas a la base de datos se cierren de forma automática al terminar de ejecutarse el bloque de código. Gracias a esto, se eliminan pérdidas de memoria sin necesidad de usar llamadas explícitas en bloques finally.

Finalmente, se ampliaron las funciones de la clase ExtrasDAO mediante el desarrollo de métodos específicos como listarAviones(), listarAeropuertos() y listarEstadosVuelos(). Estas rutinas se encargan de realizar consultas generales en las tablas, recorrer los resultados del ResultSet para guardar los registros en estructuras dinámicas de tipo ArrayList y devolver los datos ordenados en arreglos de tipo String[], listos para ser mostrados de manera limpia en los componentes de selección de la interfaz gráfica.

Fecha 09/10/2026

Durante esta jornada se expandió significativamente la capa de persistencia y control transaccional del proyecto Chorifly mediante el desarrollo e integración de nuevos métodos colectores en la clase especializada ExtrasDAO, acoplándolos operativamente con el diseño e implementación de la interfaz gráfica VentanaCrudBoletos. El objetivo central fue dar soporte

seguro a la gestión, procesamiento y selección guiada de entidades vinculadas al alta, consulta y edición de pasajes.

A nivel de arquitectura de software, la clase ExtrasDAO fue robustecida a través del desarrollo de funciones colectoras especializadas que aíslan la lógica SQL de los componentes de presentación, resolviendo de forma segura el acceso a tablas vinculadas estrictamente al módulo de pasajes. El método listarServicios se incorporó para interrogar al motor relacional mediante consultas parametrizadas sobre la tabla Servicio, procesando el cursor ResultSet mediante un mapeo objeto-relacional manual que instancia objetos de la clase entidad Servicio y puebla exhaustivamente sus atributos de identificación, nombre, precio y descripción para ser devueltos en una colección dinámica tipada. En consonancia, la rutina listarPasajeros fue añadida para extraer secuencialmente el atributo del nombre del pasajero desde la persistencia relacional hacia un contenedor dinámico, mientras que la función colector listarVuelos se diseñó para consultar el identificador numérico e implementar una conversión explícita mediante la sentencia String.valueOf(), unificando los tipos de datos en cadenas de texto antes de incorporarlos a la colección.

Por otra parte, se maquetó y programó la lógica funcional para la interfaz VentanaCrudBoletos, la cual aprovecha las extensiones de la capa DAO para mitigar errores de inserción por parte de los operadores del sistema. A nivel de comportamiento de software, se estructuró un mecanismo de selección guiada mediante ventanas emergentes inyectando eventos de escucha de clics de mouse sobre los campos de texto no editables de pasajero y vuelo. Al activarse, estos componentes interceptan la acción del usuario, consumen las listas dinámicas de la capa de persistencia y las transforman en arreglos nativos, desplegando

cuadros de diálogo interactivos mediante JOptionPane para forzar al operador a seleccionar exclusivamente registros válidos en el motor de datos.

Asimismo, se desarrolló un algoritmo iterativo para la sincronización e inyección dinámica de checkboxes dentro del evento de selección de servicios que consume los objetos provistos por el colector correspondiente. El sistema instancia componentes de verificación parametrizados con los costos y nombres en tiempo de ejecución, los apila verticalmente dentro de un panel configurado con distribución de diseño en eje vertical y realiza una comprobación de paridad con los estados de selección previos para preservar la memoria volátil de las elecciones del usuario antes de desplegar la ventana de diálogo final de tipo mensaje plano.

Fecha 12/10/2026

Durante esta jornada se agregó el estado Programado a los estados posibles de un vuelo dentro de la base de datos de Chorifly. Esta modificación se realizó para poder definir con precisión qué vuelos estarían disponibles al momento de comprar un boleto desde el programa Java.

La lógica prevista para el CRUD de compra de boletos consiste en que el usuario seleccione un vuelo de la lista y, a partir de esa selección, el sistema identifique automáticamente el avión asignado a dicho trayecto. Sin embargo, para que un vuelo aparezca como una opción elegible al comprar un boleto, debe encontrarse obligatoriamente en estado Programado. De esta manera, se evita mostrar vuelos que no estén habilitados para esta operación.

Esta decisión también estableció una limitación en la organización de los itinerarios: cada avión puede tener como máximo un único vuelo en estado Programado a la vez. Esto resulta necesario porque los asientos están asociados directamente al objeto Avion y no al objeto Vuelo.

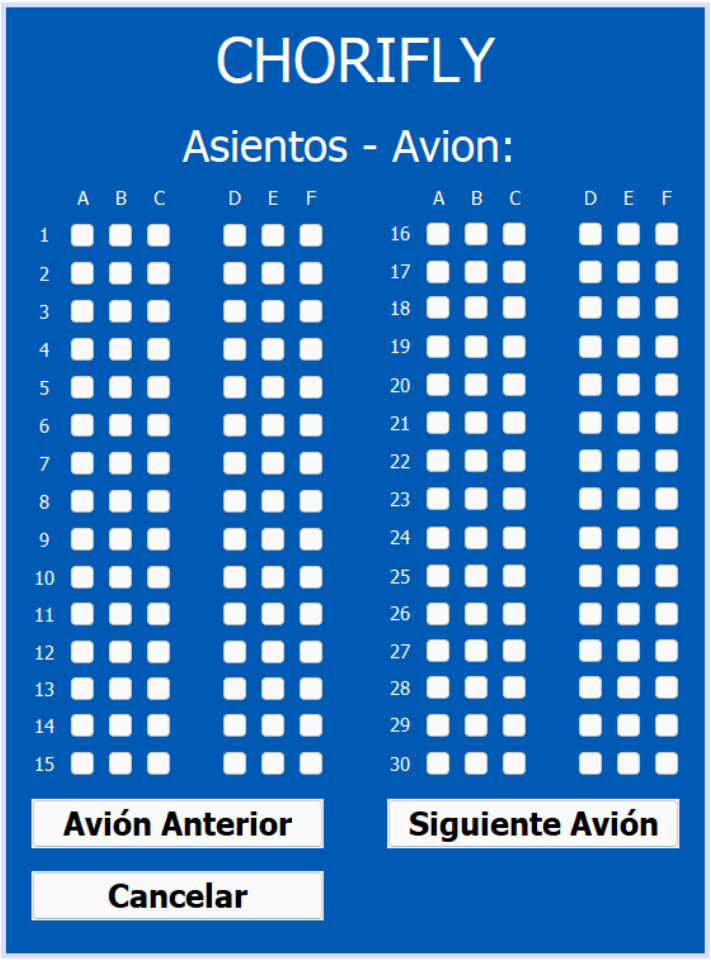
Si un mismo avión tuviera varios vuelos programados al mismo tiempo, aparecerían exactamente los mismos asientos disponibles para viajes diferentes, lo que generaría graves conflictos y duplicaciones durante la compra de boletos. Con este cambio, el estado Programado permitió conectar la disponibilidad de los vuelos con la selección de asientos, manteniendo una organización coherente entre los vuelos, las aeronaves y sus ubicaciones físicas.

En cuanto al desarrollo del entorno gráfico, se avanzó de forma significativa en el diseño del CRUD de boletos, el cual quedó preparado para su finalización en las próximas sesiones. En paralelo, se trabajó en la vista de selección de asientos dentro de VentanaAsientos.

Por el momento, la interfaz permite elegir el avión del cual se quiere revisar el asiento. Sin embargo, esta función se modificó para que próximamente sea eliminada, logrando que el sistema cargue de forma automática y exclusiva los asientos del avión que corresponda al vuelo seleccionado por el pasajero.

A nivel de reglas de negocio en la interfaz, se programó el flujo de control para que el usuario solo pueda interactuar con el mapa de asientos si tiene activo el servicio adicional de Selección de Asiento. Para dar más realismo y orden al programa, se colocaron títulos formales a todas las ventanas desarrolladas hasta la fecha.

Además, se comenzó a programar la lógica de asignación aleatoria de asientos para aquellos pasajeros que no compren el servicio adicional, y se planificó el bloqueo definitivo para impedir la selección de boletos en vuelos que no tengan el estado de planificación requerido.



Fecha 13/10/2026

Durante esta jornada se completó la maquetación y el desarrollo de la interfaz gráfica de administración mediante la creación de las pantallas VentanaCrudCheckIn y VentanaCrudPagos. Con la incorporación de estos dos últimos componentes, el sistema de Chorifly cuenta ahora con todas las ventanas necesarias para que el usuario administrador

pueda gestionar de manera integral el ciclo operativo de los vuelos, los asientos, los boletos, los pagos y el registro de embarque de los pasajeros.

A nivel de comportamiento del programa, la clase VentanaCrudPagos fue diseñada para controlar los flujos de caja y las transacciones comerciales del sistema. La pantalla monta una tabla principal JTable conectada a la capa de persistencia mediante los métodos de PagoDAO, permitiendo listar los comprobantes emitidos de forma directa. Asimismo, se integraron componentes para filtrar los datos según el estado del pago y campos de texto bloqueados para visualizar el costo de los servicios adicionales y el monto total cobrado, garantizando que el operador pueda auditar la integridad de los datos financieros.

Por otra parte, se programó la lógica visual de VentanaCrudCheckIn para dar soporte al proceso de registro en el aeropuerto y asignación de pases de abordar. La interfaz gráfica se estructuró para cargar de manera dinámica los datos de las entidades relacionadas de Vuelo, Pasajero y Boleto, facilitando la trazabilidad del operador.

Se incluyeron cuadros de diálogo a través de JOptionPane para confirmar el alta exitosa del registro de check-in en el motor de base de datos y se enlazaron eventos de selección para asignar de forma manual la puerta de embarque correspondiente a cada itinerario.

Finalmente, se ajustó el flujo de navegación de la aplicación enlazando estos nuevos formularios con la pantalla del menú de administración VentanaMenuPrincipalAdmin. Al igual que en las interfaces desarrolladas anteriormente, se programaron los eventos de los botones para que al abrir cualquiera de los dos nuevos paneles de gestión se destruya de forma limpia la ventana anterior utilizando el método dispose(), asegurando un uso óptimo de

la memoria volátil y evitando conflictos de superposición de pantallas dentro del sistema gráfico.

Fecha 14/10/2026

Durante esta jornada se terminaron de desarrollar y poner en funcionamiento las pantallas de administración VentanaCrudBoletos, VentanaCrudCheckIn y VentanaCrudPagos. Con estas implementaciones se corrigieron la mayoría de los errores lógicos y de interfaz gráfica detectados en las pruebas previas, logrando que el programa sea completamente funcional en su módulo de gestión. Para el correcto funcionamiento operativo del sistema en esta etapa, se definieron un conjunto de inserciones de datos provisionales dentro de los scripts de la base de datos relacional.

A nivel de comportamiento del programa, las pantallas se conectaron con sus respectivos controladores para asegurar el flujo de los datos comerciales. Los métodos de acción ahora capturan y validan de forma correcta las operaciones de alta y consulta, permitiendo que las tablas de la interfaz gráfica muestren los registros actualizados en tiempo real y que las alertas de JOptionPane respondan de manera adecuada ante cada evento.

Por otra parte, se llevó a cabo una limpieza integral en la clase utilitaria ExtrasDAO con el fin de optimizar el código fuente. Se eliminaron varias funciones de búsqueda que se habían vuelto redundantes debido a la evolución del diseño del software. Todas las llamadas a estos métodos repetidos fueron reemplazadas de forma directa por sus equivalentes nativos dentro de las clases de persistencia estándar de la aplicación.

A partir de esta refactorización, las consultas específicas de selección pasaron a ser gestionadas de forma centralizada por componentes como PasajeroDAO y PagoDAO, entre otros. Este cambio redujo de forma significativa el acoplamiento entre las ventanas gráficas y las clases accesorias, mejorando la modularidad y facilitando el mantenimiento a futuro del código del proyecto Chorifly.

Fecha 16/10/2026

Durante esta jornada se llevó a cabo una reestructuración profunda en el módulo de asignación de plazas mediante el desarrollo y puesta en funcionamiento de la nueva interfaz gráfica VentanaSeleccionAsientos. Este componente reemplaza de forma definitiva a la versión anterior de la pantalla de asientos, mejorando la disposición visual del mapa de ocupación y optimizando los eventos de selección interactiva para los usuarios del sistema Chorifly.

A nivel de comportamiento del programa, la clase VentanaSeleccionAsientos se programó para cargar dinámicamente el listado simétrico de las ubicaciones del avión. La interfaz utiliza una matriz visual construida con componentes Swing que simulan la distribución física de las aeronaves por filas y columnas. Se incorporó una rutina interna que consulta el estado real de cada plaza en la base de datos a través de la capa AsientoDAO, permitiendo pintar de colores diferentes aquellos asientos que ya se encuentran ocupados o vendidos para un vuelo en particular.

Por otra parte, se programaron las reglas de negocio vinculadas a los servicios adicionales del boleto. El sistema valida si el pasajero tiene contratado el servicio de Selección de Asiento antes de permitirle hacer clic sobre los casilleros del mapa visual. Si el servicio no está activo, el programa bloquea de forma automática la interacción con la matriz de botones y activa un algoritmo de asignación aleatoria, el cual busca un asiento libre dentro de los bloques disponibles y lo vincula al pasaje sin la intervención del usuario.

Finalmente, se ajustó el flujo de navegación de las interfaces para acoplar esta nueva ventana con el proceso general de compra gestionado en VentanaCrudBoletos. Al abrirse la pantalla de selección de asientos, se envían como parámetros los objetos activos de Pasajero y Vuelo correspondientes para asegurar la integridad referencial de los datos de ocupación, destruyendo de forma segura el formulario anterior mediante el método dispose() para liberar recursos de la memoria dinámica.

Fecha 18/10/2026

Durante esta jornada se realizaron dos modificaciones adicionales en la base de datos del proyecto Chorifly para dar soporte a la gestión del personal. En primer lugar, se agregó el campo correspondiente al teléfono dentro de la tabla Empleado, ya que se había omitido en el diseño inicial y resultaba necesario para almacenar la información de contacto de los trabajadores.

Por otra parte, se incorporaron registros provisionales mediante sentencias de inserción para cargar datos de prueba en las tablas del motor relacional. El objetivo de este cambio fue disponer de información real en el sistema para evaluar los métodos de consulta y listado

desarrollados en el entorno Java. Con esto se evitan errores provocados por la falta de registros y se permite comprobar que las tablas visuales puedan recuperar y mostrar los datos correctamente.

En paralelo con estas actualizaciones de la persistencia, se diseñó y programó la interfaz gráfica VentanaCrudEmpleados dentro del paquete del proyecto. Esta pantalla se estructuró para centralizar el alta, la baja y la modificación de la nómina de trabajadores de la aerolínea, enlazando sus componentes Swing directamente con la nueva estructura de la base de datos que ya incluye el campo de teléfono.

A nivel de comportamiento del programa, la nueva ventana de empleados se conectó con la clase EmpleadoDAO para canalizar las operaciones de guardado y lectura de datos. Se programaron los eventos correspondientes para capturar el texto ingresado en los formularios y cargarlo en una lista JTable, cerrando de forma limpia la pantalla actual mediante la instrucción dispose() al regresar al menú principal de administración.

Fecha 19/10/2026

Durante esta jornada se concluyó formalmente la construcción y puesta a punto de la capa gráfica de usuario del proyecto Chorifly mediante el diseño e implementación de la pantalla VentanaCrudUsuarios. Con la adición de este componente, el sistema alcanza su estado definitivo de madurez funcional y visual, dejando el programa completamente operativo en todas sus interfaces de administración antes de iniciar la migración hacia las rutinas internas del motor de base de datos.

A nivel de comportamiento del programa, la interfaz VentanaCrudUsuarios fue programada para centralizar el control de accesos, credenciales y privilegios de los operadores del sistema. La ventana monta una tabla principal JTable vinculada de forma directa con la capa de persistencia a través de los métodos de UsuarioDAO. Esto permite listar en tiempo real las cuentas registradas, modificar contraseñas y activar o desactivar las banderas lógicas de privilegios de administrador o empleado normal.

Por otra parte, esta sesión marcó el cierre de la etapa de desarrollo de software basada en consultas embebidas dentro del entorno Java. El programa se consolidó como una versión final completamente estable y funcional, donde todos los módulos CRUD (usuarios, empleados, pasajeros, vuelos, asientos, boletos, check-in y pagos) interactúan de forma limpia y coordinada, controlando las reglas de negocio a través de la clase Validaciones y emitiendo alertas visuales mediante JOptionPane.

Finalmente, se realizó una auditoría general sobre los flujos de navegación conectados desde las pantallas VentanaMenuPrincipal y VentanaMenuPrincipalAdmin. Se verificó que todas las transiciones destruyan de manera correcta los marcos gráficos anteriores mediante el método dispose() para optimizar el uso de los recursos de la memoria dinámica, dejando el repositorio de código fuente limpio y preparado para la siguiente fase del proyecto, la cual consistirá en la fusión y delegación de la lógica relacional hacia procedimientos almacenados y triggers en el servidor SQL.

Fecha 26/10/2026

Durante esta jornada se organizó formalmente el control de versiones de todo el proyecto Chorifly mediante la implementación de la convención técnica de Versionado Semántico. Se modificaron los nombres de todos los archivos y carpetas del desarrollo utilizando el formato estructurado AA.B.C, donde el primer dígito representa los cambios mayores, el segundo los avances menores y el tercero las correcciones de errores.

Esta reorganización resolvió un problema en el ordenamiento jerárquico de los directorios del programa. Anteriormente, al usar un listado simple, el sistema ordenaba de forma incorrecta las carpetas, posicionando la versión veinte antes que la entrada número tres.

Al aplicar este estándar de versionado semántico, se aseguró que las versiones menores se mantengan correctamente ordenadas de forma correlativa, facilitando el mantenimiento del código fuente y el seguimiento histórico de la aplicación.

Fecha 30/10/2026

Durante esta jornada se inició de forma exitosa la primera etapa de migración de las consultas SQL embebidas hacia procedimientos almacenados en el servidor de la base de datos de Chorifly. El trabajo se centró en intervenir el componente ExtrasDAO, modificando la lógica interna de los métodos de búsqueda que alimentan a los diferentes formularios de administración del sistema.

A nivel de arquitectura de software, se eliminaron por completo del código Java aquellas cadenas de texto extensas que contenían sentencias de consulta directas como SELECT o uniones complejas entre tablas. En su lugar, estas instrucciones fueron reemplazadas por llamadas estructuradas utilizando la sintaxis de CALL Procedimiento().

Este cambio optimiza el rendimiento de la aplicación al delegar la ejecución de las consultas al motor relacional, desacopla la persistencia del código de Java y eleva los estándares de seguridad de Chorifly al prevenir inyecciones de código malicioso.

Fecha 02/11/2026

Se modificaron las restricciones en claves foráneas, sustituyendo NOT NULL por NULL en las tablas del histórico (Pasajero, Empleado, Boleto, Pago y CheckIn). Esta configuración, combinada con la propiedad ON DELETE SET NULL, previene fallos por integridad referencial al eliminar usuarios o empleados, garantizando la persistencia de los registros contables y de auditoría sin comprometer la base de datos.

Asimismo, se completó la migración de las consultas SQL embebidas hacia procedimientos almacenados en las clases AsientoDAO, CheckInDAO, EmpleadoDAO, PagoDAO, PasajeroDAO, UsuarioDAO y VueloDAO, reemplazando las sentencias directas por llamadas estructuradas mediante CALL Procedimiento(). Para asegurar la compatibilidad y legibilidad del sistema, se conservó el uso de PreparedStatement en el código de Java, se aislaron todas las rutinas de la base de datos con bloques DELIMITER individuales y se unificó la nomenclatura de los parámetros bajo el estándar p_.

Fecha 04/11/2026

Con el fin de optimizar la integridad referencial y flexibilizar la desvinculación de registros en la base de datos Chorifly, se reestructuró su modelo lógico, permitiendo valores nulos (`NULL`) en claves foráneas que anteriormente estaban definidas como `NOT NULL`. Este ajuste permite evitar errores relacionados con las restricciones de integridad referencial y facilita la eliminación de registros en entidades principales, preservando la consistencia de la base de datos.

Por otra parte, se unificó la capa de persistencia mediante la integración de los procedimientos almacenados desarrollados conjuntamente por Joaquín Ferramola, Gonzalo Delgado y quien suscribe. La lógica se estandarizó siguiendo el modelo implementado en `ExtrasDAO`, transformando las consultas SQL previamente embebidas en dicha clase en procedimientos almacenados. Este mismo criterio fue aplicado por Gonzalo Delgado en los componentes vinculados a HTML.

Por último, se integraron los triggers desarrollados por Gonzalo Delgado, disponibles en el archivo `Chorifly BD (Con Triggers).sql`. Inicialmente, se utilizaba la versión `Chorifly BD (Sin Triggers).sql` para evitar excepciones en Java ocasionadas por triggers provisionales que debieron implementarse manualmente. Un ejemplo de este conflicto se presentaba durante la emisión de un boleto: tanto la aplicación Java como la base de datos intentaban registrar automáticamente un pago con estado «Pendiente». Como consecuencia, se generaban dos registros de pago para un mismo boleto, lo que provocaba un conflicto con las restricciones de unicidad de la base de datos.

Al intentar solucionar esta problemática, surgió una dificultad adicional durante la creación de boletos, relacionada con la implementación de `BoletoDAO`. Además de generar el registro correspondiente al pago, la operación debía insertar los servicios contratados en la tabla intermedia `Boleto_Servicio`. Para realizar esta última operación era necesario disponer del identificador del boleto recién creado, lo que inicialmente representaba un inconveniente dentro del mecanismo de inserción existente.

Para resolverlo, se realizó una revisión integral del módulo de gestión de boletos (`BoletoDAO`), con el objetivo de adaptar su funcionamiento a los procedimientos almacenados y triggers incorporados en la base de datos. Durante este proceso, se modificó el mecanismo de creación de boletos, reemplazando `PreparedStatement` por `CallableStatement`, debido a la necesidad de recuperar el identificador generado mediante un parámetro de salida (`OUT`) del procedimiento almacenado. Esto permitió obtener el ID del boleto recién creado y utilizarlo posteriormente para registrar los servicios asociados en la tabla intermedia `Boleto_Servicio`.

Asimismo, se revisó la lógica de edición de boletos, verificando que tanto los datos principales como las relaciones con los servicios adicionales se actualizaran correctamente. Se decidió mantener la gestión de dichas relaciones desde Java, debido a su profunda integración con la estructura existente de `BoletoDAO`. De esta manera, se evitó realizar una modificación extensa del módulo para trasladar toda esa responsabilidad a triggers.

Finalmente, se corrigió un error en `VentanaCrudBoletos` que impedía seleccionar asientos al editar un boleto existente. Si bien los servicios contratados se mostraban correctamente marcados en sus respectivos `JCheckBox`, la aplicación no reconocía la contratación del

servicio «Selección de Asiento». Mediante pruebas de depuración, se identificó que la comprobación de este servicio se encontraba dentro de un recorrido que podía interrumpirse anticipadamente mediante una instrucción `break`. Se reorganizó dicha lógica para separar la identificación de los servicios contratados de la comprobación específica del servicio de selección de asiento. Además, se sincronizó el estado de las variables booleanas utilizadas por los distintos eventos de la interfaz, solucionando finalmente el inconveniente.